双目标柔性作业车间调度问题算法设计（makespan + TWTE）

> 算法代号：**DBC-NSGA-II**（Due-date-driven Bidirectional Constructive NSGA-II；中文释义：交货期导向的双向构造式 NSGA-II —— 阶段一为交货期感知双向构造初始化（§5，零迭代），阶段二为改进 NSGA-II（双向邻域 + 自适应算子，§6–§8），两阶段评估口径见 §9）
>
> 文档版本：v1（供思路审阅）
>
> 优化目标：`min f1 = makespan`（最大完工时间） `min f2 = TWTE`（总提前 + 拖期，无权）

---

1. 问题定义与数学模型

1.1 问题描述

柔性作业车间调度问题（FJSP）中：

- 有 n 个工件 $J = \{J_1, \ldots, J_n\}$、m 台机器 $M = \{M_1, \ldots, M_m\}$；
- 工件 J_i 含 h_i 道工序 $O_{i,1}, O_{i,2}, \ldots, O_{i,h_i}$，工序之间满足**先后约束**（$O_{i,j}$ 完成后 $O_{i,j+1}$ 才能开工）；
- 每道工序 $O_{i,j}$ 有一台**可选机器集合** $M_{ij} \subseteq M$，在机器 $M_k \in M_{ij}$ 上的加工时间为 p_{ijk}（随机器不同而异）；
- 需要为每道工序（1）**选择一台机器**（路由子问题），并（2）在每台机器上**决定各工序的加工顺序**（排序子问题）。

> **重要前提（已确认）**：同一工件 J_i 的**不同工序可以分配到同一台机器**。即不存在"每个工件在每台机器上至多加工一次"的限制，这是经典 FJSP（Brandimarte / Kacem / Hurink 标准定义），而非部分文献里的受限变体。同机时的先后仅由 precedence 约束保证，两工序在同一机器上串行排布即可。

1.2 符号约定

符号	含义
$n,\ m$	工件数、机器数
h_i	工件 J_i 的工序数
O_{ij}	工件 J_i 的第 j 道工序
M_{ij}	O_{ij} 的可选机器集合
p_{ijk}	O_{ij} 在机器 M_k 上的加工时间（$k \in M_{ij}$）
d_i	工件 J_i 的交货期
$WT_i = WE_i = 1$	拖期/提前取等权（本方案采用无权 E+T 之和）
$S_{ij},\ C_{ij}$	O_{ij} 的开始时间、完工时间
$C_i = C_{i,h_i}$	工件 J_i 的完工时间（其最后一道工序的完工）
$T_i,\ E_i$	工件 J_i 的拖期、提前量
$y_{ijpqk} \in \{0,1\}$	机器 k 上的排序变量：O_{ij} 先于 O_{pq} 时为 1（仅当二者都分配到机器 k 时约束生效）

总工序数记为 $O_{total} = \sum_i h_i$。

1.3 目标函数

目标一：makespan（最小化最大完工时间）

$$\min\ f_1 = C_{max} = \max_{i,j}\ C_{ij} \tag{1}$$

目标二：TWTE（最小化总提前量与拖期量之和，无权）

$$\min\ f_2 = TWTE = \sum_{i=1}^{n}\left(T_i + E_i\right) \qquad (\text{无权形式，即 } WT_i = WE_i = 1) \tag{2}$$

$$T_i = \max\left(0,\ C_i - d_i\right), \qquad E_i = \max\left(0,\ d_i - C_i\right) \tag{3}$$

其中工件级拖期/提前量由**该工件最后一道工序的完工时间** C_i 与交货期 d_i 的差决定（与文献 [Nessari et al. 2024] 一致）。

1.4 约束条件

1. **机器唯一分配**：每道工序恰好分配到一台可选机器

$$\sum_{k \in M_{ij}} x_{ijk} = 1, \quad \forall i, j \tag{4}$$

2. **加工时长**：

$$C_{ij} - S_{ij} = \sum_{k \in M_{ij}} p_{ijk} \cdot x_{ijk}, \quad \forall i, j \tag{5}$$

3. **工序先后约束**（同一工件内）：

$$S_{i,j+1}\ \geq\ C_{ij}, \quad \forall i,\ j = 1, \ldots, h_i - 1 \tag{6}$$

4. **机器容量约束**（同一台机器上的任意两道工序不重叠）。对分配到同一台机器 M_k 的任意两道工序 O_{ij}、O_{pq}，引入 0-1 排序变量 y_{ijpqk}：

$$S_{ij} \geq C_{pq} - L \cdot (1 - y_{ijpqk}) - L \cdot (2 - x_{ijk} - x_{pqk}), \quad S_{pq} \geq C_{ij} - L \cdot y_{ijpqk} - L \cdot (2 - x_{ijk} - x_{pqk}) \tag{7}$$

其中 L 为充分大的常数。式 (6) 与 (7) 共同保证：即使同一工件的两道工序落在同一台机器上，也按先后顺序串行加工（precedence 与 non-overlap 自然兼容，无需额外约束）。

1.5 关键特性：两个目标在时间轴上的相反张力（本设计的总纲）

- **makespan** 要求所有工序**尽可能早、尽可能紧凑（左移、不放空）**，目标是把"最晚完工点"往左压缩；
- **TWTE** 要求每个工件**恰好在其交货期 d_i 完成（右移对齐）**：早了产生早到惩罚 E_i，晚了产生拖期惩罚 T_i（无权 E+T 之和）。

由此得到三条直接影响算法设计的结论：

> **结论 A**：TWTE 端的最优解**必须允许机器插入空闲（idle）**，把过早完工的工件右移到 d_i 附近以消除早到惩罚——这是 makespan 优化完全不涉及的自由度；
>
> **结论 B**：Pareto 前沿必然**跨度大、且局部非凸**，用加权和（weighted sum）会漏解，故采用**支配（非支配排序）**机制；
>
> **结论 C**：单一"左移（active）解码"会系统性**高估 TWTE**（把所有工件压成尽早完工 → 早到惩罚爆表），因此同一染色体必须用**两套解码**分别评估两个目标（见 §3.2）。

---

2. 算法总体框架

```
┌──────────────────── 阶段一：交货期感知双向构造初始化（DABI） ────────────────────┐
│  前向构造（makespan 端）+ 反向 JIT 构造（TWTE 端）+ 得分导向扰动 + 随机补充   │
│  → 目标空间网格覆盖（每格留代表）+ 拥挤距离补满 → 初始种群 P0                  │
│  （makespan 端 / TWTE 端 / 中间 trade-off 全覆盖，零迭代）                     │
└──────────────────────────────────────────────────────────────────────────────┘
                                      ↓
┌──────────────────────── 阶段二：主搜索（改进 NSGA-II） ──────────────────────┐
│  · 双段编码 MS+OS；左移解码算 f1、右移空闲插入解码算 f2                        │
│  · 交叉：OS 用 POX、MS 用 UX；变异：动态位数（创新⑤）                         │
│  · 非支配排序 + 拥挤距离（目标先 min-max 归一化）+ 精英保留                   │
│  · 自适应局部搜索（创新④）：成功率驱动轮盘赌选算子，迭代 ≥80% 才触发          │
│    邻域算子含"左移类"与"右移类"（创新③），min-max 逐代更新                    │
└──────────────────────────────────────────────────────────────────────────────┘
                                      ↓
┌──────────────────── 阶段三：输出（随机口径 + 长仿真） ──────────────────────┐
│  对非支配前沿 P* 做 R=10000 次蒙特卡洛长仿真，报告随机口径期望与稳健性统计    │
└──────────────────────────────────────────────────────────────────────────────┘
```

> **两阶段评估口径（§9）**：阶段一用**固定（名义）加工时间**构造种子（§5）；阶段二切换到**随机加工时间**做蒙特卡洛评估（式 18），进入阶段二时按 §9.3 重建 min/max 基座。阶段三仅剩"取随机口径非支配前沿 + 可选长仿真校验"（§9.5）。

---

3. 编码与解码

3.1 双段整数编码（主编码，贯穿阶段二与局部搜索）

采用经典 `MS + OS` 两段式，总长度 $L_{MS} = L_{OS} = O_{total}$：

- **MS（机器选择段）**：`MS[a]` 表示第 a 道工序在其可选机器集 M_{ij} 中的**下标**（$0 \le MS[a] \le |M_{ij}| - 1$）。第 a 道工序按"工件主序、工序次序"编号，即 a 对应 O_{ij}。
- **OS（工序排序段）**：工件号的排列，工件 i 出现恰好 h_i 次；第 k 次出现的 i 表示 J_i 的第 k 道工序。

> **同机允许**由 MS 天然满足：不同工序（含同一工件的不同工序）在 MS 中独立取值，选到同一台机器不冲突；解码器按 precedence 把同机同工件的工序串行排布，无需修复。

3.1.1 具体示例（3 工件 × 2 机器）

设 $n = 3$, $m = 2$：J_1 两道工序、J_2 两道工序、J_3 一道工序，则 $O_{total} = 2 + 2 + 1 = 5$。各工序的可选机器集与加工时间如下（"第 0 台 / 第 1 台"即该工序可选机器集内两台机器的下标顺序）：

工序 O_{ij}	可选机器集 M_{ij}	第 0 台机器（p）	第 1 台机器（p）
O_{11}	{M1, M2}	M1（3）	M2（4）
O_{12}	{M1, M2}	M1（5）	M2（2）
O_{21}	{M1, M2}	M1（2）	M2（4）
O_{22}	{M1, M2}	M1（5）	M2（1）
O_{31}	{M1, M2}	M1（3）	M2（3）

工序按"工件主序、工序次序"编号：
$a = 1 \to O_{11}$，$a = 2 \to O_{12}$，$a = 3 \to O_{21}$，$a = 4 \to O_{22}$，$a = 5 \to O_{31}$。

（1）MS 段（长度 5，逐位为第 a 道工序选机器下标）

```
MS = [ 0 , 1 , 0 , 0 , 0 ]
       ↑   ↑   ↑   ↑   ↑
     O11 O12 O21 O22 O31
```

- `MS[1]=0` → O_{11} 选它可选机器集的第 0 台 = **M1**；
- `MS[2]=1` → O_{12} 选第 1 台 = **M2**；
- `MS[3]=0` → O_{21} 选第 0 台 = **M1**；
- `MS[4]=0` → O_{22} 选第 0 台 = **M1**（**与 O_{21} 同机**，体现"同作业可同机"）；
- `MS[5]=0` → O_{31} 选第 0 台 = **M1**。

（2）OS 段（长度 5，工件号排列，工件 i 出现 h_i 次，第 k 次出现即其第 k 道工序）

```
OS = [ 1 , 2 , 1 , 3 , 2 ]
       └第1次出现=O11
           └第1次出现=O21
               └第2次出现=O12
                   └第1次出现=O31
                       └第2次出现=O22
```

即加工顺序（OS 扫描序）为：$O_{11} \to O_{21} \to O_{12} \to O_{31} \to O_{22}$。

（3）左移解码（§3.2a，逐工序摆到各自机器上最早可行时刻）：

顺序	工序	机器	时长	前道工序完工	本机已占用到	开始 S	完工 C
1	O_{11}	M1	3	—（首道）	0	0	3
2	O_{21}	M1	2	—	3	3	5
3	O_{12}	M2	2	O_{11}→3	0	$\max(3,0)=3$	5
4	O_{31}	M1	3	—	5	5	8
5	O_{22}	M1	5	O_{21}→5	8	$\max(5,8)=8$	13

得到 $f_1 = C_{max} = \max C_{ij} = 13$；机器甘特（M1 上 O_{21}、O_{22} 同属 J_2，由 precedence 串行、二者间被 O_{31} 隔开）：

```
M2:              [ O12 ]        (3-5)
M1: [ O11 ][ O21 ][ O31 ][   O22    ]
    0     3     5     8            13
```

3.2 双解码：左移（算 makespan）与右移（算 TWTE）

同一染色体（MS+OS）用**两套解码**分别得到两个目标值，是落实结论 C 的核心机制：

（a）左移解码（半主动 / append-only：按 OS 顺序排到该机当前可用时刻，不主动回填机器空档）→ 计算 $f_1 = C_{max}$

```
对 OS 中每个工序 O_ij（按出现顺序）：
    候选开始 = max( 该工件前道工序完工 C_{i,j-1},  机器 M_k 当前可用时刻 )
    S_ij = 候选开始;   C_ij = S_ij + p_ijk
    更新机器 M_k 可用时刻 = C_ij；工件 i 当前工序指针 +1
```

得到的 $f_1 = \max C_{ij}$。

（b）右移解码（idle 插入）→ 计算 $f_2 = TWTE$

在**固定 MS 与每台机器上的工序顺序**的前提下（由左移解码确定），做一次**右移传播**，把各工件的完工时间尽量贴到 d_i、在不增加任何拖期的前提下消除早到：

```
右移传播（RightShiftRepair，固定 MS 与各机工序顺序）：
输入：左移调度(各工序 S,C)、交货期 d_i
1. 给每个工序设"最晚完工" LC(o):
   · o 是工件 i 的最后一道工序 → LC(o) = max( C(o), d_i )
       早到(C<d):  目标贴到 d_i
       拖期/准点:  保留 C(o)，不进一步推后(避免增大 T_i)
   · 否则 → LC(o) = +∞
2. 按完工时间从后向前，逐工序收紧 LC(o):
   LC(o) = min( LC(o),
                LC(工件后继) − p(工件后继),   # 无工件后继则 +∞
                LC(机器后继) − p(机器后继) )  # 无机器后继则 +∞
3. 右移后的开工 = LC(o) − p(o)   (在最晚可行处开工，空闲自然插入)
4. 工件 i 完工 C_i = 其最后一道工序的 LC;
   E_i = max(0, d_i−C_i),  T_i = max(0, C_i−d_i),  f_2 = Σ(T_i + E_i)   # 无权
```

> **规则**：f_1 取自左移解码（makespan 的真实最优必然左移紧凑）；f_2 取自右移解码（TWTE 需要空闲对齐交货期）。二者解耦，避免一个解码同时拖累两个目标。右移传播保证**不增加任何 tardiness**（只消除 earliness），因此它对 TWTE 单调不增；但右移会把有松弛的末道工序推到 $d_i \gg C_{max}$，右移后调度的 C_{max} 可能变大——(f_1, f_2) 不对应同一张甘特图，这是"解耦口径"（§1.5 结论 C）。

右移解码示例（延续 §3.1.1 的 3 工件 × 2 机器算例）

左移调度与完工（§3.1.1 已得）：

```
M2:              [ O12(2) ]                    (3–5)
M1: [ O11(3) ][ O21(2) ][ O31(3) ][  O22(5)  ]  (0–13)
```

设交货期 $d_1 = 8,\ d_2 = 12,\ d_3 = 8$（无权，$WT_i = WE_i = 1$）。左移下各工件完工 $C_1 = 5,\ C_2 = 13,\ C_3 = 8$：

$$TWTE_{\text{左移}} = T_1 + E_1 + T_2 + E_2 + T_3 + E_3 = (13 - 12) + (8 - 5) = 1 + 3 = 4$$

（J_2 拖期 1，J_1 早到 3。）

右移传播步骤：

1. **初始化 LC**（只对工件最后一道工序）：
   - O_{12}（J_1 末道，$C = 5 < d_1 = 8$）$\rightarrow LC = \max(5, 8) = 8$；
   - O_{22}（J_2 末道，$C = 13 > d_2 = 12$，已拖期）$\rightarrow LC = \max(13, 12) = 13$（**不再推后**）；
   - O_{31}（J_3 末道，$C = 8 = d_3$）$\rightarrow LC = \max(8, 8) = 8$。
2. **从后向前收紧**（按完工时间降序）：

工序	初值 LC	工件后继约束	机器后继约束	最终 LC
O_{22}	13	无	无（M1 末位）	**13**
O_{31}	8	无	$LC(O_{22}) - p = 13 - 5 = 8$	**8**
O_{12}	8	无	无（M2 独用）	**8**
O_{21}	$+\infty$	$LC(O_{22}) - 5 = 8$	$LC(O_{31}) - 3 = 5$	**5**
O_{11}	$+\infty$	$LC(O_{12}) - 2 = 6$	$LC(O_{21}) - 2 = 3$	**3**

3. **右移开工** = $LC - p$：

```
右移：  M2:                 ··[ O12 ]          (6–8，3–6 处空闲插入)
       M1: [ O11 ][ O21 ][ O31 ][  O22  ]     (不变：被已拖期的 J2 顶死)
```

只有 O_{12} 从 $3-5$ 右移到 $6-8$（M2 插入 3 单位 idle）；M1 上 4 道工序因 O_{22} 已拖期、O_{31} 已准点而被"钉住"，全部不动。

4. **重算完工与 f_2**：$C_1 = 8,\ C_2 = 13,\ C_3 = 8$，

$$TWTE_{\text{右移}} = T_2 = 13 - 12 = 1$$

结果：TWTE 从 $4 \rightarrow 1$（J_1 的早到 3 被空闲插入消除），而 f_1 仍取左移的 $C_{max} = 13$、J_2 的拖期 1 未增加。

> **机制小结**：右移只在"有松弛"的末道工序（本例独占 M2 的 O_{12}）上插入 idle；被拖期工件顶住的机器（M1）不会被推后，故**不产生新的或更大的拖期**。这正是 §1.5 结论 C 要求的"TWTE 端需要 idle 自由度"，也是双解码（§3.2）得以成立的原因。

3.3 右移解码在迭代中的作用

右移解码**不是搜索算子，而是 f_2 的评估器**——每次被调用只"算值"，不产生新个体、不改编码。但它是 TWTE 方向不被评估失真带偏的关键，贯穿每一次评估：

- **阶段一构造初始化**：DABI 的每个候选解都用双解码得 (f_1, f_2)（§5.9）；若启用可选精修（§5.15），其每次接受判定同样走双解码；
- **阶段二 NSGA-II**：初始种群、每代交叉变异后子代、非支配排序、拥挤距离，f_2 全用右移解码；
- **邻域算子判定**：§7.4 候选解都经双解码评估；§8.1 的"成功 = f_2 下降 / 非支配改进"也依赖它。

其三个具体作用：

1. **让支配关系在正确的 TWTE 尺度上进行**：非支配排序比较 (f_1, f_2)。若 f_2 用左移解码算，那些"本可插入空闲对齐交货期"的解会被算出一个虚高 TWTE，进而在支配比较中被误判为更差、被淘汰——这正会重演要修的"TWTE 端塌陷"。右移解码把这个偏差抹掉。
2. **让右移类算子的改进能被正确度量**：R1 换慢机/顺序后移（§7.3）等右移操作的效果要落到 f_2 数值上才算"成功"（§8.1）。没有右移解码，右移省下的早到惩罚无法反映到目标值里，成功率统计与轮盘赌（§8.2）就失去意义。
3. **它是"解耦评估"的右半边（创新②）**：f_1 永远取左移（紧凑真实最优）、f_2 永远取右移（idle 对齐）。同一染色体、两套解码分别兑现，避免"一套紧凑调度同时拖累两个目标"。

与 R1 算子的分工（易混，但层级不同）：

	右移解码（§3.2b）	**R1 邻域算子（§7.3）**
性质	评估机制，被动投影	搜索算子，主动改动
是否改编码	否	产生新候选解
是否进轮盘赌	否	是（§8）
触发时机	每次评估	局部搜索阶段

一句话关系：**右移解码负责"在固定编码下把 f_2 用最优右移算出来"，R1 负责"通过改 MS/OS 把右移对齐结构固化进编码"**——一个在评估层、一个在搜索层，配合工作，不重复。

---

4. 动态 min-max 归一化

4.1 逐代动态 min-max（已定稿，覆盖"离线固定"旧方案）

对两个目标各维护**到目前为止的全局极值**，每代进化后刷新：

$$
\begin{aligned}
f_1^{min}(t) &= \min\big(f_1^{min}(t-1),\ \min_{x\in P_t} f_1(x)\big), \qquad f_1^{max}(t) = \max\big(f_1^{max}(t-1),\ \max_{x\in P_t} f_1(x)\big),\\
f_2^{min}(t) &= \min\big(f_2^{min}(t-1),\ \min_{x\in P_t} f_2(x)\big), \qquad f_2^{max}(t) = \max\big(f_2^{max}(t-1),\ \max_{x\in P_t} f_2(x)\big)
\end{aligned}
\tag{8}
$$

- 初值：$f^{min} = +\infty$，$f^{max} = -\infty$；
- 单调性：f^{min} 单调不增、f^{max} 单调不减；
- **全局共享**：阶段一内部共用；进入阶段二按 §9.3 用随机口径重建基座后继续单调更新；
- **顺序约定**：阶段一先造两端——反向构造（§5.4）撑开 f_2 端、前向构造（§5.3）撑开 f_1 端，再由几何覆盖（§5.8）补中间区，使极值尽快有宽度。

为什么极值必须全局共享：

共享一份全局极值，等于给整个目标空间套一个统一的 $[0,1]^2$ 坐标系；同一代内所有依赖"尺度"的比较（拥挤距离、精英保留）都用这一套坐标，密度才可比。若逐代各自统计 min/max（按本代种群重算），同一目标在不同代会被压缩或拉伸，拥挤距离与精英保留随之失真。

反例：某一代种群在 makespan 上只覆盖很窄的区间（如 $80-82$）、TWTE 区间却很宽；另一代反过来。若逐代各自归一化，前者会把 makespan 的 $80-82$ 拉伸成满 $[0,1]$，后者把同一批 makespan 值压成一小段——跨代尺度不一致，算出的"密度"不再可比，精英保留会系统性偏向某一类解。

> **一致性副作用**：极值全局单调膨胀，同一固定解的归一化坐标会随代数漂移；因排序与拥挤距离都只在**代内**比较、跨代不排序，故该漂移无副作用。

归一化（每代用当前极值重新计算）：

$$\bar{f}_1(x)=\frac{f_1(x)-f_1^{min}}{f_1^{max}-f_1^{min}+\varepsilon},\qquad \bar{f}_2(x)=\frac{f_2(x)-f_2^{min}}{f_2^{max}-f_2^{min}+\varepsilon},\qquad \varepsilon=10^{-6} \tag{9}$$

> **除零保护**：目标在该代若 $f^{max}-f^{min}<\varepsilon$（尚只有单一取值），则该目标归一化值取 0。

> **实现约定（归一化的存放位置）**：因极值逐代漂移，归一化只在比较的当下一代现场计算；任何跨代存续的结构（种群、精修候选、外部存档）都应**存原始目标值** (f_1,f_2)**，在比较/排序的当下一代用当前** f^{min}/f^{max} **现场归一化**，绝不缓存已归一化的值（详见 §4.1 与 §14 R1）。这样每次比较两边同尺度，杜绝"旧尺度 vs 新尺度"的错配。
>
> **归一化用途边界**：归一化仅用于（i）阶段一可选短程精修的接受判定（§5.15）与（ii）阶段二拥挤距离的尺度对齐；**支配关系本身不需要归一化**（Pareto 支配与尺度无关）。其中（ii）的跨度来源用 §4.1 的全局逐代极值（式 (8)(9)），而非每个前沿的局部极值，理由及示例见 §6.3.1。

---

5. 阶段一：交货期感知双向构造初始化（创新①）

本节算子记为 **DABI**（Due-date-Aware Bidirectional Initialization，**交货期感知双向构造初始化**）：Due-date-Aware 指反向构造由 d_i 倒排开工时刻、前向规则按段选取；Bidirectional 指前向（压 makespan 端）与反向（压 TWTE 端）两个方向同时构造；Initialization 指零迭代、一次调用直接产出初始种群 P_0。

5.1 动机与定位

`可尝试的方法.md` 记录的现象：用 makespan 单目标预热后，NSGA-II 的结果在 TWTE 端变差、Pareto 前沿"不对称塌陷"（makespan 端好、TWTE 端缺解/无右移）。

本节改为**零迭代的构造式初始化**：不再使用任何"随机起点 + 迭代收敛"的求解器，而是**按方向直接构造**高质量解。三点理由：

1. **随机起点在双目标下无人负责两端**：任何"随机起点 + 单目标/加权收敛"的求解器，落点由目标函数决定、起点却是随机的，"两端种子"要靠多跑几组权重加运气才能铺到；构造式初始化把"覆盖两端"变成显式约束（§5.8）。
2. **构造解的质量下限已接近经典 dispatch + 账本路由的水平**，而"随机起点 + 迭代收敛"的预热链路要用约 5×10^3 次解码才能把它"爬"到同一水平，性价比不高。
3. **只有纯离散构造、没有实值编码与连续更新**，实现量与调参量都明显下降（也不存在连续→离散的漂移隐患）。

> **保留的核心主张（创新① 不变的部分）**：多方向覆盖——makespan 端、TWTE 端、中间 trade-off 区**都必须有种子**，否则 TWTE 端塌陷会重演。改变的只是实现方式：从"各方向分别跑一个迭代求解器"改为"多方向各造种子 + 目标空间几何覆盖"。
>
> 离散短程精修（§5.15）保留为**可选增强**，主链路不依赖它。

5.2 路由段：双账本 GS-θ

对工序 O_{ij}，构造时同时维护两个账本：

账本	含义	服务的方向
$R[m]$	机器 m 的最早可用时刻（前向构造，初值 0）	makespan
$L[m]$ / $\mathrm{occ}[m]$	反向构造账本：v1 为单值最晚可用时刻 $L[m]$；v2 为每机不重叠已占区间表 $\mathrm{occ}[m]$（见 §5.4.2）	TWTE
$\mathcal{J}(m)$	已排到 m 上的**工件**集合 → 跨工件耦合度 $\varphi_{ij}(m)$	TWTE

选机打分（前向构造用 R；反向构造用 L 并改为"尽量晚开工"，见 §5.4）：

$$\mathrm{score}_{ij}(m) = R[m] + \theta\, p_{ijm} + \gamma\, \bar{p}\, \varphi_{ij}(m) \tag{10}$$

$$\varphi_{ij}(m) = \frac{|\mathcal{J}(m)| - \mathbf{1}\big(i \in \mathcal{J}(m)\big)}{n-1} \tag{11}$$

符号含义（式 (10)(11) 中出现的全部量）

符号	含义	取值 / 量纲
O_{ij}	工件 J_i 的第 j 道工序，$i=1,\dots,n$，$j=1,\dots,h_i$	—
M_{ij}	O_{ij} 的可选机器集合	M 的非空子集
m	当前被评估的候选机器，遍历 M_{ij}	—
p_{ijm}	O_{ij} 在机器 m 上的加工时间	时间，>0
$R[m]$	**前向账本**：机器 m 的最早可用时刻（其上已排工序全部完工的时刻）	时间，≥ 0，初值 0
$L[m]$ / $\mathrm{occ}[m]$	**反向账本**：v1 为机器 m 的最晚可用时刻；v2 为不重叠已占区间表（§5.4.2）	时间；初值 $H=\max_i d_i+\max_i(h_i\,\bar{p}_{\mathrm{job},i})$（§5.4）
θ	加工时间权重：在"再等一会儿"与"自身加工更久"之间权衡	$[0,1]$，建议档 $\{0,\ 0.5,\ 1\}$
γ	解耦强度：跨工件耦合惩罚的倍率	$[0,1]$，建议档 $\{0,\ 0.5,\ 1\}$
$\bar{p}$	全算例平均加工时间（对全部可用 (i,j,m) 三元组取均值）；作用是把无量纲的 φ 换算回时间	时间
$\mathcal{J}(m)$	当前已排入机器 m 的工件集合，**按工件去重**（同一工件的多道工序只算一次）	集合，$\lvert\mathcal{J}(m)\rvert\leq n$
$\varphi_{ij}(m)$	若把 O_{ij} 排到机器 m 上而引入的跨工件耦合度	$[0,1]$，无量纲
$\mathbf{1}(i\in\mathcal{J}(m))$	指示函数：$i\in\mathcal{J}(m)$ 取 1，否则取 0；作用是把"本工件自己"从耦合里剔除	$\{0,1\}$
n	工件数	正整数
$\mathrm{score}_{ij}(m)$	选机打分：前向**越小越优先**；反向用式 (12b) 的 score 取最大（§5.4）	时间

> **三项的量纲**：$R[m]$ 与 θp_{ijm} 都是时间；φ 无量纲，乘 $\bar{p}$ 后变为 $[0,\ \gamma\bar{p}]$ 的时间，故三项可直接相加。除 $n-1$ 是归一化——机器 m 上最多容纳 $n-1$ 个**别的**工件，φ 的上限恰为 1、与算例规模无关，因此同一个 γ 在 10×6 与 20×15 上含义一致。

- $\theta=0$：最早可用优先 → 主动调度、紧凑，**makespan 端**；
- $\theta\in\{0.5,1\}$：兼顾加工时间（≈ GS 方法里 $T+m$ 的 $T+p$ 语义）；
- $\gamma=0$：不做解耦，**makespan 端**；$\gamma\in\{0.5,1\}$：解耦，**TWTE 端。**

> **解耦项的作用**：f_2 由右移解码（§3.2b）给出，其质量上限取决于"给定 MS+OS 后还能插入多少 idle"，而 idle 只能插在机器有松弛处。若一台机器被多个工件挤满，右移空间为零。$\gamma\varphi$ 就是"把工件链尽量分到不同机器上"的量化，等于**把右移自由度直接买进编码。**解耦必然伤负载均衡——这个冲突正是两端种子的来源，也解释了为什么 γ 必须按方向分档而不能全局统一。

5.2.1 走查示例：同一算例在 $\gamma=0$ 与 $\gamma=1$ 下的选机差异

（1）算例与预处理量

工序	可选机器与加工时间
O_{11}（J1）	M1=2, M2=4
O_{12}（J1）	M2=3, M3=5
O_{21}（J2）	M1=6, M3=2
O_{22}（J2）	M1=3, M2=4, M3=6
O_{31}（J3）	M2=2, M3=3
O_{32}（J3）	M1=4, M3=5

共 13 个可用 (i,j,m) 三元组，加工时间之和 49，故 $\bar{p}=49/13\approx 3.77$，$\gamma=1$ 时 $\gamma\bar{p}=3.77$；$n=3$ 故 $n-1=2$，φ 只能取 $\{0,\ 0.5,\ 1\}$。构造顺序 J1→J2→J3、工件内按工序序；每步取 score 最小者（并列按 p 小、再按机号小）；时间窗按 $S_{ij}=\max\big(C_{i,j-1},\ R[m]\big)$ 推进。

（2）$\theta=0.5,\ \gamma=0$（makespan 端；第三项恒为 0，表中只写 $R+\theta p$）

步骤	工序	M1	M2	M3	选中	时间窗 S–C
1	O_{11}	$0+0.5\cdot 2=\mathbf{1}$	$0+0.5\cdot 4=2$	—	M1	0–2
2	O_{12}	—	$0+1.5=\mathbf{1.5}$	$0+2.5=2.5$	M2	2–5
3	O_{21}	$2+3=5$	—	$0+1=\mathbf{1}$	M3	0–2
4	O_{22}	$2+1.5=\mathbf{3.5}$	$5+2=7$	$2+3=5$	M1	2–5
5	O_{31}	—	$5+1=6$	$2+1.5=\mathbf{3.5}$	M3	2–5
6	O_{32}	$5+2=\mathbf{7}$	—	$5+2.5=7.5$	M1	5–9

（3）$\theta=0.5,\ \gamma=1$（TWTE 端；耦合项为 $3.77\varphi\in\{0,\ 1.89,\ 3.77\}$）

步骤	工序	M1	M2	M3	选中	时间窗 S–C
1	O_{11}	$0+1+0=\mathbf{1}$	$0+2+0=2$	—	M1	0–2
2	O_{12}	—	$0+1.5+0=\mathbf{1.5}$	$0+2.5+0=2.5$	M2	2–5
3	O_{21}	$2+3+1.89=6.89$	—	$0+1+0=\mathbf{1}$	M3	0–2
4	O_{22}	$2+1.5+1.89=5.39$	$5+2+1.89=8.89$	$2+3+0=\mathbf{5.00}$	**M3（翻转）**	2–8
5	O_{31}	—	$5+1+1.89=\mathbf{7.89}$	$8+1.5+1.89=11.39$	M2	5–7
6	O_{32}	$2+2+1.89=\mathbf{5.89}$	—	$8+2.5+1.89=12.39$	M1	7–11

（4）两个解的机器甘特

$\gamma=0$（M1 = 9、M2 = 5、M3 = 5，$C_{max}=9$；M1 上 J1→J2→J3 连排，无空档）：

```
M2:      [ O12 ]                (2–5)
M3: [ O21 ][ O31 ]              (0–5)
M1: [ O11 ][ O22 ][  O32   ]    (0–9)
```

$\gamma=1$（M1 = 11、M2 = 7、M3 = 8，$C_{max}=11$；J2 整体移到 M3，M1 中段让出 5 单位空档）：

```
M2:      [ O12 ][ O31 ]          (2–7)
M3: [ O21 ][   O22   ]           (0–8)
M1: [ O11 ]        [  O32  ]     (0–2，空档 2–7，7–11)
```

(5) 四个要点

1. **第 4 步的翻转就是 γ 的全部意义**：O_{22} 在 M1 上只要 3、在 M3 上要 6，$\gamma=0$ 时当然选 M1（3.5 < 5.0）；$\gamma=1$ 时 M1 被罚 $3.77\times0.5=1.89\to5.39$，而 M3 因 $\mathcal{J}(M3)=\{J2\}$ 且 O_{22} 也属 J2 $\to\varphi=0$ 不罚 $\to$ 5.00，于是选了加工时间翻倍的 M3。这就是"**把右移自由度直接买进编码**"的字面运算：花 3 个时间单位的加工代价，买下"整台机器只有自己"。代价也立刻显形：J2 的工件链被推到 M3 上（$C=8$），J3 的 O_{31} 只能等 M2 空出来才能开工（$S=5$），C_{max} 从 9 升到 11——**这正是必须同时保留 $\gamma=0$ 种子的原因。**
2. **指示函数 $\mathbf{1}(i\in\mathcal{J}(m))$ 的作用，看第 4 步的 M3**：此时 $\mathcal{J}(M3)=\{J2\}$、当前工序 O_{22} 也属 J2，故 $\varphi_{22}(M3)=\frac{1-1}{2}=0$（不是 0.5）。若漏掉该项，同一工件的后续工序会"嫌自己挤"而到处跳机，与 §1.1「同一工件的不同工序可以分配到同一台机器」的前提直接冲突。反观第 3 步的 M3：$\mathcal{J}(M3)=\varnothing$，$\varphi=0$，同样是"干净"的意思。
3. **量纲与归一化**：见 §5.2 正文"三项的量纲"注——R、θp 是时间，$\gamma\bar{p}\varphi$ 乘上 $\bar{p}$ 后也是时间，三项才能直接相加；除 $n-1$ 使 φ 上限为 1，与算例规模无关。
4. **两端种子天然不同**：同一个构造器只改 γ，就得到两套路由（9/5/5 对 11/7/8）与两种工件分布（$\mathcal{J}$ 为 {J1,J2,J3}/… 对 {J1,J3}/{J1,J3}/{J2}）。这既支撑 §5.8 用 (θ,γ) 网格铺点、由 §5.4 反向构造强化 TWTE 端的做法，也印证 §5.13 风险表中"γ 过大拖坏 makespan"的机制。

> **本例的边界**：只演示路由段（选机 + 时间窗推进），OS 直接取构造次序，**未做右移解码**，因此不给出 f_2 数值。真实的 f_2 与"空档能否被右移利用"见 §3.2b 的右移传播。

5.3 构造器 A：前向调度构造（makespan 端 + 中间区）

标准的 active schedule 逐步生成：

```
输入: θ, rule, γ=0, 随机种子
R[m] ← 0, C_ij ← 0, J(m) ← ∅, done ← ∅
while done ≠ 全部工序:
    Rls ← { O_ij : 同工件前道工序已完成 }
    O_ij ← 按 rule 在 Rls 中取前 2 名随机（并列块用 r_tie 打乱）
    m* ← argmin_{m∈M_ij} score_ij(m)   // 式(10)，取前 2 名随机
    S_ij ← max(C_{i,j-1}, R[m*]);  C_ij ← S_ij + p_ijm*    // 约定 C_{i,0} = 0
    R[m*] ← C_ij;  J(m*) ← J(m*) ∪ {i};  done ← done ∪ {O_ij}
输出: MS（m* 在 M_ij 内的下标）, OS（调度次序）, S
```

随机化规格（伪代码里两处"取前 2 名随机"的精确含义）：

1. 每步仍**只排一道工序、只放一台机器**；"前 2 名"是**优集**，不是"前两名都排"。
2. **工序决策**：把 `Rls` 按当前 `rule` 的**优先级分值**升序排定——统一约定"分值越小越优先"，故 LPT 用 $-\bar{p}_{ij}$、AVPRO 用 $-\sum_{j'\ge j}\bar{p}_{ij'}$；**分值相等的连续块（并列块）先用独立随机流 `r_tie` 打乱**，再取前 2 名等概率随机取一（随机流 `r1`）；$|\mathrm{Rls}|=1$ 时直接取，$|\mathrm{Rls}|=2$ 时即 50/50。
3. **选机决策**：把 M_{ij} 按 $\mathrm{score}_{ij}(m)$ 升序排定；并列块同样用 `r_tie` 打乱，再取前 2 台等概率随机取一（随机流 `r2`）；$|M_{ij}|=1$ 时直接取。
4. 三条随机流**各自独立**（`r_tie` 打乱并列块、`r1` 定工序、`r2` 定机号），均自同一随机种子派生；固定种子则整条构造完全可复现。
5. 这是 GRASP 受限候选表（RCL）的**固定大小**版本：RCL 恒为 2，偏离幅度可预期；候选集只有 1 个元素时自动退化为确定性贪心。
6. **为什么并列随机是安全的**：并列意味着规则（或打分）**无法区分**这些候选，任取其一不降低质量下界；无并列时第 2、3 条自然退化为确定性破平局。反之，并列块很大说明该规则在此算例上区分

力弱——这既是多样性来源，也是"这条规则几乎没信息"的诊断信号（可把并列块大小记为统计量）。

排序规则按方向分两组（文献中的混合初始化把 6 条规则混用于"破平局"，这里提升为**全局优先规则**并按方向分组）：

规则里的 $\bar{p}_{ij}$ 表示工序 O_{ij} 在其**可选机器上的平均加工时间** $\bar{p}_{ij}=\frac{1}{|M_{ij}|}\sum_{m\in M_{ij}}p_{ijm}$——SPT/LPT 直接按它排序，AVPRO/CR/S/RO 用剩余工序的 $\sum_{j'\geq j}\bar{p}_{ij'}$。注意与 §5.2 的**全局**均值 $\bar{p}$（式 (10) 里把 φ 换算成时间的那个量）区分：前者每道工序一个值、随工序变化，后者是全算例一个常数。

组	规则	含义
makespan 组	SPT	最短加工时间优先
	LPT	最长加工时间优先（配合 SPT 增加跨度）
	AVPRO	优先安排剩余工作量（$\sum_{j'\geq j}\bar{p}_{ij'}$）最大的工件
交期组	EDD	按 d_i 升序
	CR	最小松弛 $\max\big((d_i-S_{ij})/\sum_{j'\geq j}\bar{p}_{ij'},\,0\big)$
	S/RO	剩余松弛/剩余工序数，取最小者

5.3.1 走查示例：`rule = SPT`、$\theta=0.5$、$\gamma=0$

沿用 §5.2.1 的算例，只演示"每步都取前 2 名中的第 1 名"这条**确定性路径**，便于逐行核对；取第 2 名的效果见（5）。示例为便于核对，并列按工序号**确定性**破平局；实现时按随机化规格第 2 条用 `r_tie` 打乱并列块。

（1）预处理：工序级平均加工时间

工序	M_{ij} 与 p_{ijm}	$\bar{p}_{ij}$
O_{11}（J1）	M1=2, M2=4	3.00
O_{12}（J1）	M2=3, M3=5	4.00
O_{21}（J2）	M1=6, M3=2	4.00
O_{22}（J2）	M1=3, M2=4, M3=6	4.33
O_{31}（J3）	M2=2, M3=3	2.50
O_{32}（J3）	M1=4, M3=5	4.50

$\gamma=0$ 故式 (10) 第三项恒为 0，打分即 $R[m]+0.5\,p_{ijm}$。

（2）逐步构造

步	已排	`Rls`（附 $\bar{p}_{ij}$）	rule 前 2 名	选中	选机打分（$R+0.5p$）	时间窗 S–C
1	$\varnothing$	O_{11}(3)、O_{21}(4)、O_{31}(2.5)	O_{31}、O_{11}	O_{31}	M2 $0{+}1.0=\mathbf{1.0}$；M3 $0{+}1.5=1.5$	0–2
2	O_{31}	O_{11}(3)、O_{21}(4)、O_{32}(4.5)	O_{11}、O_{21}	O_{11}	M1 $0{+}1.0=\mathbf{1.0}$；M2 $2{+}2.0=4.0$	0–2
3	$+O_{11}$	O_{12}(4)、O_{21}(4)、O_{32}(4.5)	O_{12}、O_{21}（并列按工序序）	O_{12}	M2 $2{+}1.5=3.5$；M3 $0{+}2.5=\mathbf{2.5}$	2–7
4	$+O_{12}$	O_{21}(4)、O_{32}(4.5)	O_{21}、O_{32}	O_{21}	M1 $2{+}3.0=\mathbf{5.0}$；M3 $7{+}1.0=8.0$	2–8
5	$+O_{21}$	O_{22}(4.33)、O_{32}(4.5)	O_{22}、O_{32}	O_{22}	M1 $8{+}1.5=9.5$；M2 $2{+}2.0=\mathbf{4.0}$；M3 $7{+}3.0=10.0$	8–12
6	$+O_{22}$	O_{32}(4.5)	仅 O_{32}	O_{32}	M1 $8{+}2.0=10.0$；M3 $7{+}2.5=\mathbf{9.5}$	7–12

第 1 步把 O_{31} 放到 M_2 后，$R[M_2]=2$；第 2 步 O_{11} 在 M_1、M_2 上的打分因此变成 $0+1.0$ 与 $2+2.0$——**账本对选机的影响在这里第一次显形。**

> **打分为什么用 $R[m]$ 而不是 $\max\big(C_{i,j-1},R[m]\big)$**：$C_{i,j-1}$ 对所有候选机器是同一个值，但 $\max$ 会把"都早于 $C_{i,j-1}$"的机器压成同一个数、丢掉区分度；用 $R[m]$ 保留"哪台机器更早空出来"的信息，真实开工时间放到 $S=\max(\cdot)$ 一步体现。

（3）输出与甘特

```
       0    2         5         7        8        12
M1:  [ O11 ][       O21 (6)              ]
M2:  [ O31 ]                             [ O22 (4) ]
M3:        [ O12 (5)            ][ O32 (5)         ]
```

- $MS=[0,1,0,1,0,1]$：按工序编号 $O_{11},O_{12},O_{21},O_{22},O_{31},O_{32}$ 排列，值为 m^* 在 M_{ij} 内的**下标**（如 $O_{12}\to M_3$，而 $M_{12}=\{M_2,M_3\}$，故取 1）；
- $OS=[3,1,1,2,2,3]$：工序被安排的次序（每个工件出现 $h_i=2$ 次）；
- $f_1=C_{max}=12$（O_{22} 收尾），f_2 需 §3.2b 右移解码，本例不展开。

（4）解码复核：构造即左移解码

按 OS 扫描序重跑 §3.1 的左移解码：

序	工序	机器	$S=\max\left(C_{i,j-1},R[m]\right)$	C
1	O_{31}	M2	$\max(0,0)=0$	2
2	O_{11}	M1	$\max(0,0)=0$	2
3	O_{12}	M3	$\max(2,0)=2$	7
4	O_{21}	M1	$\max(0,2)=2$	8
5	O_{22}	M2	$\max(8,2)=8$	12
6	O_{32}	M3	$\max(2,7)=7$	12

与构造过程得到的 S **逐个相同**。所以 f_1 在构造时已推出，不必再解码——这正是 §5.3 比"随机串 + 解码"快的来源。

（5）对照：再取一次"二选一"的效果

先澄清 RCL（$k=2$）的含义：**每一步都在前 2 名里等概率抽一个**，而不是"只有并列时才随机"。规则（SPT/EDD/...）只负责排出前 2 名；`r_tie` 打乱并列块只决定**谁进入前 2 名**，不改变"总从前 2 名里抽一个"这一点。因此"取第 2 名"是任何一步都可能发生的合法偏离，与是否并列无关。

确定性路径（每步取第 1 名、选机取打分最小的第 1 台）给出 $C_{max}=12$。对每一步分别做**单次**偏离（只在该步取第 2 名，其余步仍取第 1 名）：

偏离步	取第 2 名工序	取第 2 台机器
1	12	10
2	**7**	13
3	**7**	**7**
4	13	14
5	12	12
6	12	12

可见：

- 单次偏离既可变差（13/14），也可**明显变好**——步 2、步 3 取第 2 名工序，或步 3 取第 2 台机器，都得 $C_{max}=7$。

结论：6 步里每步至多 2 选 1（第 6 步只剩 1 个候选，路径数上限 $2^5=32$），C_{max} 可落到 7/10/12/13/14 等值——list scheduling 对"工序顺序 + 选机"高度敏感；而"前 2 名等概率"把每一步的偏离限制在**只差一个名次**的有界范围内，既**提供多样性又不至于乱序**。注意这只是 SPT/$\theta=0.5$/$\gamma=0$ 下的展示，不能外推成"第 2 名总是更差"。

（6）三个要点

1. **为什么固定 $\gamma=0$**：前向构造只产出 active（尽早）解、天然没有 idle，此时加解耦惩罚只会把工序推向空闲的慢机、纯伤 makespan；右移自由度由 §5.4 反向构造负责（$\gamma=1$、区间账本取最晚可行开工、容差内抽签，见 §5.4.2）。
2. **"中间区"从哪来**：交期组规则（EDD/CR/S/RO）只改"先排哪个工件"，不改"紧凑性"，所以它们产出的是**紧凑但向 d_i 靠拢**的解——TWTE 只能靠 §3.2b 右移解码部分兑现，正好落在两端之间。

3. **复杂度**：每步 $|Rls| \le n$ 次规则比较 + $|M_{ij}| \le m$ 次打分 $\rightarrow O\big(O_{total}(n+m)\big)$。MK03（150 道工序、8 台机）约 3.5K 次比较、单次构造远低于 0.1 ms，故 §5.8 能一次铺出几十上百个种子。

5.4 构造器 B：反向 JIT 构造（TWTE 端主力）

思路：**从交货期倒排**。前向构造只能"尽早"，而 TWTE 端恰恰需要"恰好"。倒排天然把各工件钉在 d_i 附近。

```
输入: θ, rule, γ, 工件序 order, 选机容差 slack, 随机种子        // T 段 γ=1；中间区取 γ∈{0,0.5}
H ← max_i d_i + max_i(h_i · p̄_job,i)                  // 时间地平线（p̄_job,i 为工件 i 各工序 p̄_ij 的均值）
occ[m] ← ∅  (m=1..M)                                  // 每台机器一张"不重叠已占区间表"，取代单值 L[m]
按 order 定工件处理序：LST_i = d_i − chain_min_i 降序（默认），或 due_desc / edd（消融用）
for 工件 i（按该序），j 从 h_i 递减到 1:
    LC ← (j == h_i ? d_i : S_{i,j+1})                 // 后继的最晚开工 = 本道最晚完工
    对每个 m∈M_ij:
        λ_ijm ← min(LC, H) − (1 + θ)·p_ijm            // 逐机量：留出 θ 的提前裕度，且必须在 LC 前完工
        s(m) ← latest_start(occ[m], λ_ijm, p_ijm)     // 区间账本：λ 之前、长 p 的最晚空闲起点；无空档 → ⊥
        score(m) ← s(m) − γ·p̄·ϕ_ij(m)                // γ 项只在可行机器之间做偏好排序
    F ← { m∈M_ij : s(m) ≠ ⊥ }                         // 可行机器集（允许填空档）
    if F ≠ ∅:
        best ← max_{m∈F} score(m)
        m* ← 在 { m∈F : score(m) ≥ best − slack·p̄_ij } 中等概率抽一台    // p̄_ij 为本工序平均加工时间
        S_ij ← s(m*);  C_ij ← S_ij + p_ijm*
        occ[m*] ← occ[m*] ∪ {[S_ij, C_ij)}             // 只有真正放下的才占位
        J(m*) ← J(m*) ∪ {i}                            // ϕ 账本只随成功工序更新
    else:                                              // 真正放不下：计 fail，但不再"毒化"账本
        fail++; 记 fail_ops
        m* ← argmax_m λ_ijm（λ 最靠右的一台）; S_ij ← 0
        // 不写入 occ[m*]、不修改任何账本（含 J）；失败工序的 MS 仍按该机确定，会进入解码
// ── OS 回写：失败工序的 S=0 只是贴边记录，不能直接按 (S,a) 排序 ──
ok  ← 非失败工序按 (S, a) 升序
bad ← 失败工序按工件分组（组内按工序序号升序），再做"工件轮转序" rt（逐轮每工件各取一道）
OS ← 把 rt 的第 k 个工序插到 ok 的第 ⌈k·|ok|/|bad|⌉ 个位置之前（= |ok| 时追加到末尾）
输出: MS, OS, S（S 仍不可当调度，必须重新解码）
```

$$LC_{ij} = \begin{cases} d_i, & j = h_i \\ S_{i,j+1}, & j < h_i \end{cases} \tag{12}$$

$$\begin{aligned} \ell_m(p,\lambda) &= \max\{\, s \ge 0 :\ s \le \lambda,\ [s, s+p) \cap \mathrm{occ}[m] = \varnothing \,\}, \\ \mathrm{score}_{ij}(m) &= \ell_m\big(p_{ijm},\ \lambda_{ijm}\big) - \gamma \bar{p}\ \varphi_{ij}(m), \qquad \lambda_{ijm} = \min\big(LC_{ij}, H\big) - (1+\theta)\, p_{ijm} \end{aligned} \tag{12b}$$

> **v2 说明（式 (12b)）**：把单值账本 $L[m]$ 换成"每台机器一张不重叠已占区间表 $\mathrm{occ}[m]$"，ℓ_m 返回"λ 之前、长 p 的最晚空闲起点"（无空档为 $\perp$）；m^* 在可行集 $F = \{m : \ell_m \neq \perp\}$ 中按 score 取最大，并在与最大值差 $\le \mathrm{slack} \cdot \bar{p}_{ij}$ 的机器间等概率抽签——给 TWTE 端保留随机性，否则同组合重试只能得到重复解。单值账本 $L[m]$ 只是 $\mathrm{occ}[m] = [L[m], \infty)$ 的退化情形：只能整体左移、

> 不能填机器空档，这是 v1 fail 率高的主因（§5.4.2）。

> θ **在反向构造里的含义：提前裕度。**正向的 θ 是"加工时间权重"（式 (10)），反向的 θ 是同一个 p 上的**安全余量**：$(1+\theta)p$ 让工序比"刚好赶得上"再早 θp 落位。
>
> - $\theta = 0$：最贴交货期（$S = \ell_m(p,\ \min(LC, H) - p)$），JIT 最紧，也最容易被挤到 `fail`；
> - $\theta = 1$：提前整整一个 p 落位，机器上留出的空档最多（右移解码的余地最大），代价是早到量上升；
> - 由于 $C = S + p \le \min(LC, H) - \theta p \le \min(LC, H)$，$\theta \ge 0$ 时**不会违反"赶得上后继"的硬约束**，只是把工序整体左移；$\theta = 0$ 即式 (12) 的原始形式。
>
> 这也让 §5.8 的 (θ, γ) 网格在反向上真正张开：同一个 $\gamma = 1$、不同 θ 给出"紧贴型"与"留空型"两类 TWTE 种子，反向上 θ 不再是被传进来却不用的空参数。

> **为什么它是 TWTE 端主力**：反向解自带**结构性 idle**，右移解码几乎不用再动，f_2 起点最低；且按 LST 降序先让交期晚的工件占右侧、交期紧的再填左侧（§5.4.2），天然避免"多个早到工件互相挤占机器"。
>
> **失败处理（v2）**：所有候选机器都 $\lambda_{ijm} < 0$ 或找不到空档时才计 `fail`；`S=0` 仅作"贴边"记录，**不写入区间表、不修改任何账本**。v1 的 `L[m]\leftarrow0` 会把该机器永久封死，下一道工序必然再次失败，形成级联（实测 Mk1 第一次 fail 之后 51/51 道工序全部 fail，fail 率 0.95–0.97）；修复后 Mk1/Mk3/Mk5/Mk10 降到 0.26–0.47。注意：账本不被毒化后，级联仍会沿**工件链**发生——失败工序 $S = 0$ 经 LC 前传给前驱，使前驱 $\lambda < 0$ 而 fail。`fail` 率仍是交期紧度诊断指标：长期 >0.5 见 §5.13 的兜底顺序；残余约 0.3 属"交期地板"，如实报告即可（§5.4.2、§5.13）。

> **工件处理序（v2）**：默认按 $LST_i = d_i - \text{chain_min}_i$ **降序**（$\text{chain_min}_i = \sum_j \min_{m \in M_{ij}} p_{ijm}$），交期晚的工件先占右侧、交期紧的再填左侧，与倒排方向一致。v1 的 EDD 升序恰好相反——最紧的工件先占最右端，其余工件被整体压进 $[0, d_{\min}]$。`backward_order="edd"` 保留为消融对照。

5.4.1 走查示例：$d = (9,\ 13,\ 12)$、$\gamma = 1$、$\theta = 0$（v1：单值账本）

> 本节沿用 v1 的单值账本 $L[m]$，只用于说明账本语义、γ 的翻盘与"S 不可当调度"；**落地实现已改为区间账本 + LST 序 + 失败兜底，见 §5.4.2**（该例在新规则下的 S、fail 数会变化）。

沿用 §5.2.1 的算例并补交货期 $d_1 = 9,\ d_2 = 13,\ d_3 = 12$；全局 $\bar{p} = 3.77$、逐工件均值 $\bar{p}_{\text{job}} = (3.5,\ 4.165,\ 3.5)$，故 $\gamma\bar{p} = 3.77$、$\gamma\bar{p} \cdot 0.5 = 1.885$。
$H = \max d_i + \max_i(h_i \bar{p}_{\text{job},i}) = 13 + 2 \times 4.165 = 21.33$；EDD 序为 $J_1 \to J_3 \to J_2$。

(1) J_1（$d_1 = 9$）

工序	LC	M1	M2	M3	选中	S–C
O_{12}（末道）	$d_1 = 9$	—	$\min(21.33, 9) - 3 = \mathbf{6.0}$	$9 - 5 = 4.0$	M2	6–9
O_{11}	$S_{12} = 6$	$\min(21.33, 6) - 2 = \mathbf{4.0}$	$\min(6, 6) - 4 - 0 = 2.0$	—	M1	4–6

O_{12} 的 $\mathcal{J}(M_2) = \varnothing \Rightarrow \varphi = 0$ 不罚；O_{11} 在 M_2 上 $\varphi = \frac{1-1}{2} = 0$（自己的工件）也不罚，但 $\min(L[M_2], LC) = 6 < H$ 使其 st 更小，仍是 M_1 胜。结果 $C_1 = 9 = d_1$，**精准命中**；账本更新为 $L[M_1] = 4$、$L[M_2] = 6$。

(2) J_3（$d_3 = 12$）

工序	LC	M1	M2	M3	选中	S–C
O_{32}（末道）	$d_3 = 12$	$\min(4, 12) - 4 - 1.885 = -1.885$	—	$\min(21.33, 12) - 5 = \mathbf{7.0}$	M3	7–12
O_{31}	$S_{32} = 7$	—	$\min(6, 7) - 2 - 1.885 = 2.115$	$\min(7, 7) - 3 - 0 = \mathbf{4.0}$	M3	4–7

O_{31} 这一步是 γ 的**唯一翻盘点**：M_2 的原始 $st = 6-2 = 4.0$ 与 M_3 的 $7-3 = 4.0$ 打平，但 $\mathcal{J}(M_2) = \{J_1\} \Rightarrow \varphi = 0.5 \Rightarrow$ 罚 1.885 后跌到 2.115，于是 M_3（$\mathcal{J} = \{J_3\}$、$\varphi = 0$）胜出——"自己人挤自己人"不罚，正是解耦项的设计意图。结果 $C_3 = 12 = d_3$，再次精准命中；$L[M_3] = 4$。

(3) J_2（$d_2 = 13$）：钉不住

- O_{22}：M1 $\min(4,13) - 3 - 1.885 = -0.885$；M2 $\min(6,13) - 4 - 1.885 = 0.115$；M3 $\min(4,13) - 6 - 1.885 = -3.885$ → 取 M2，$S = 0.12$、$C = 4.12$，$L[M_2]$ 收缩到 0.12；
- O_{21}：M1 $0.12 - 6 - 1.885 = -7.77$；M3 $0.12 - 2 - 1.885 = -3.77$ → **全部为负** → `fail++`，取最大 -3.77、令 $S = 0$、$C = 2$。

为什么钉不住：三台机器的"右端"已被 J_1、J_3 占满（L 掉到 $4/6/4$），J_2 的两道工序只能往最左边挤。注意这**不是 d_2 太紧**——把 d_2 放宽到 15 仍会 `fail`，瓶颈是"右端被前两个工件占满"。

(4) 输出

> 注：本节是 v1 单值账本的**手推讲解**——每步取 st 最大者；v1 代码实跑在可行集上做 top-2 RCL 随机（同参数 seed=0…4 的 fail 分别为 2/1/4/4/2，S 各不相同），故表中 $S/OS/MS$ 不应与代码输出逐位比对。

输出项	值
S	O_{11} 4、O_{12} 6、O_{21} 0、O_{22} 0.12、O_{31} 4、O_{32} 7
OS	$[2,2,1,3,1,3]$（按 S 升序）
MS	$[0,0,1,1,1,1]$（按 $O_{11}, O_{12}, O_{21}, O_{22}, O_{31}, O_{32}$ 编号序：M1/M2/M3/M2/M3/M3 在各自可选集内的下标）

> **必须写死的口径**：`fail` 时 $C_{21} = 2 > S_{22} = 0.12$，**工件内先后已被破坏**。所以 S 只能当"排序依据 + 初值"，真正的时间窗必须由 §3.2a 左移解码重算（解码天然保 precedence）。这与 §5.3 的"构造即左移解码"不同，别混用。

(5) θ 的作用（对照）

令 $\theta = 1$（提前一个 p），J_1 的 O_{12} 打分变为 $9 - 2\times 3 = \mathbf{3.0}$（$M_2$）与 $9 - 2\times 5 = -1$（M_3）→ 仍选 M_2，$S = 3$、$C = 6$。完工比 d_1 早 3，**让出 M_2 上 3 单位空档**，但 $E_1 = 3$。θ 就是在"贴交货期（$\theta \to 0$）"与"留空档换右移余地（$\theta \to 1$）"之间做取舍。

(6) 三条实现提醒

1. S **不可直接当调度**，必须过左移解码（见上注）；
2. `fail` **率是诊断指标而非错误**：v2 下残余约 0.26 ~ 0.47 属交期地板（§5.4.2），长期 >0.5 才按 §5.13 兜底；
3. **单个反向种子会"饿死"非紧急工件**：本例 J_2 被挤到最左端。这正是 §5.8 要用 (θ, γ) 网格铺点、§5.5 要用扰动补位的直接原因。

5.4.2 落地修正（v2）：五处改动与实测

v1 在 Brandimarte 紧交期（SW 法，$T = 0.2$，$R = 1.0$）下 `fail` 率长期为 **0.95 ~ 0.97**，`segT`（TWTE 端 30 个名额）几乎产不出 JIT 种子——TWTE 端实际靠构造器 C 的补种与 D 撑着。诊断出三个叠加原因，对应五处局部改动；§5.4 的骨架（倒排 + 式 (12b) 的落位规则）不变。

(1) 三个原因

1. **单值账本"单调左压"，不允许填机器空档（主因）**：新工序必须整体排在该机器所有已排工序之前，机器时间轴只能从右往左单调生长；k 个工件要求 k 段互不重叠的链全部堆在 `[0,` 第一个放置

点] 内，窗口很快耗尽。

2. `fail` **分支"毒化"账本 → 级联**：v1 失败后把 $L[m] \leftarrow 0$，该机器上任何正加工时间的工序都必然 $st < 0$，于是下一道再 fail、再封死一台机器。实测 Mk1（EDD、$\theta = 0$、$\gamma = 0$，重复 3 次）：**第一次 fail 出现在第 4 个工序，之后 51/51 道工序全部 fail。**95% 是"第一个错误传播出去"的结果，不是真的每道工序各自不可行。
3. **工件处理序与构造方向相反**：EDD 升序让交期最紧的工件先占最右端，其余工件只能整体排到它左侧。但它只是次要因素——只改顺序 fail 仅从 0.95 降到 0.83（见下）。

消融证据（Mk1，$T = 0.2$，v1 代码实测）：

变体	fail 均值
完整 v1 实现	**0.95**
fail 时 $L[m]$ 不归零	0.95（无效：$st < 0$ 时 $L[m]$ 本已 ≈ 0）
工件序改 d_i 降序	0.83
工件序改 latest-start 降序	0.82
去掉 $L[m]$，只用工件链 LC	**0.26**
区间账本（允许塞空档）	0.34
区间账本 + 降序	**0.31**

（2）五处改动

#	改动	关键式 / 参数	说明
1	工件序：$LST_i = d_i - \text{chain_min}_i$ **降序**	`backward_order="lst"`（默认），另留 `"due_desc"`、`"edd"` 供消融	交期晚的先占右侧；收益有限，但让 JIT 语义正确
2	$L[m]$ 单值 → 每机不重叠 **区间表** $\text{occ}[m]$	式 (12b)，ℓ_m 求"最晚可行开工"	主因修复：允许塞空档（如已占 `[5,8)`、`[10,12)` 时可放 `[8,10)`）
3	`fail` 不再毒化账本	失败工序不写 $\text{occ}[m]$、不改账本	切断级联；安全性由"S 必须重新解码"保证
4	选机：可行集内取最优 + 容差抽签	`backward_slack=1.0`（$\times \bar{p}_{ij}$，本工序平均加工时间）	纯 argmax 会因重复度过高让 `segT` 只产出 4/30；纯 top-2 RCL 反而把 fail 抬到 0.52–0.55
5	失败工序的 OS 兜底	`os_fallback` 标记	失败工序 $S = 0$ 不能按 (S, a) 直接排序，否则成块挤在最前

改动 1（LST 序）——玩具实例（3 工件×2 工序，$d = [9, 13, 12]$）：

工件	各工序最短工时	chain_min	d_i	$LST = d_i - \text{chain_min}$
J_1	$\min(2,4) = 2$，$\min(3,5) = 3$	5	9	**4**
J_2	$\min(6,2) = 2$，$\min(3,4,6) = 3$	5	13	**8**
J_3	$\min(2,3) = 2$，$\min(4,5) = 4$	6	12	**6**

`lst` 降序为 $J_2(8) \to J_3(6) \to J_1(4)$：**交期最松的先占最右端**；v1 的 `edd` 升序为 $J_1 \to J_3 \to J_2$，最紧的 J_1 先占最右端，其余被压进 $[0, 9]$。

改动 2（区间账本）——"最晚可行开工"即式 (12b)，实现为每机一张按 start 升序、互不重叠的半开区间表；**首尾相接不算重叠**（`[8,10)` 可紧贴 `[6,8)` 之后放下）。判定例：

已占区间	新工序	`limit`	标量账本	区间账本
`[0,3)`、`[5,8)`	$p=2$	12	只能 $\le 0-2$ → **fail**	`[12,14)` ✓
`[5,8)`、`[10,12)`	$p=2$	11	`[3,5)`（整体压到最左，空档白白浪费）	撞 `[10,12)` 后左移到 `[8,10)` ✓
`[0,4)`、`[4,6)`、`[6,8)`	$p=2$	7	fail	撞满后 $s=-2\to$ **真 fail**（非级联）

改动 5（OS 兜底）——失败工序按工件分组（组内按工序序号升序）后取**工件轮转序** rt（逐轮每个还有失败工序的工件各取一道），再按位点 $p_k=\lceil k\cdot|ok|/|bad|\rceil$ **按比例散布**进成功序（$p_k=|ok|$ 表示追加末尾）。无失败时 OS 与改动前完全一致。同一 MS 实测：Mk8 直接按 (S,a) 排序 $f_1=1378,\ f_2=18758$，兜底后 $661,\ 6170$；Mk12 $1493/29981\to725/10284$；Mk15 $1308/26843\to696/9662$。变体对照（Mk8）：成功序 + 轮转序拼接 747/6452、轮转序前置 684/6449，**比例散布最好。**

改动 5 例子（逐位走查）——`random_instance(4, 3, min_ops=2, max_ops=3, seed=0)`，交期用 `generate_due_dates(...)×0.8` 压紧，`EDD / θ=0 / γ=0 / lst / slack=0`；11 道工序中 4 道失败。失败工序的 $S=0$ 只是"贴边记录"，**不表示先后关系**：直接按 (S,a) 排序会让同一工件的失败工序在 OS 里连续成块，左移解码退化为"逐工件串行"，f_1,f_2 为 $285/300.5$；兜底后降到 $163/199.0$。

工序 a	工件	S_a	失败
0	J_1	5.2	
1	J_1	13.2	
2	J_1	127.5	
3	J_2	66.2	
4	J_2	94.2	
5	J_2	98.2	
6	J_3	0.0	✗
7	J_3	0.0	✗
8	J_3	9.9	
9	J_4	0.0	✗
10	J_4	0.0	✗

*第 1 步：拆成两段。*成功工序按 (S,a) 升序：$ok=[0,8,1,3,4,5,2]$（$S=5.2,9.9,13.2,66.2,94.2,98.2,127.5$）；失败工序 $[7,6,10,9]$ 按工件分组、**组内按工序序号升序**：J_3：$[6,7]$、J_4：$[9,10]$（保证同一工件的失败工序内部先后不乱）。

*第 2 步：工件轮转序。*每轮从每个还有失败工序的工件各取一道：

轮次	J_3	J_4
1	6	9
2	7	10

得 $bad_rr = [6, 9, 7, 10]$。

*第 3 步：算插入位点。*第 k 个失败工序插在 ok 的第 $p_k = \lceil k \cdot n_{ok}/n_{bad} \rceil$ 个工序**之前**（$k = 0, \ldots, n_{bad} - 1$；$p_k = n_{ok}$ 表示追加末尾）。本例 $n_{ok} = 7$、$n_{bad} = 4$，位点 $[0, 2, 4, 6]$：

k	失败工序	$k \cdot n_{ok}/n_{bad}$	$\lceil \cdot \rceil$	插在
0	6	0	0	$ok[0] = 0$ 之前
1	9	1.75	2	$ok[2] = 1$ 之前
2	7	3.5	4	$ok[4] = 4$ 之前
3	10	5.25	6	$ok[6] = 2$ 之前

第 4 步：逐位合并。

```
i=0  ok[0]=0 → 先插 6          [6, 0]
i=1  ok[1]=8 → 无插入          [6, 0, 8]
i=2  ok[2]=1 → 先插 9          [6, 0, 8, 9, 1]
i=3  ok[3]=3 → 无插入          [6, 0, 8, 9, 1, 3]
i=4  ok[4]=4 → 先插 7          [6, 0, 8, 9, 1, 3, 7, 4]
i=5  ok[5]=5 → 无插入          [6, 0, 8, 9, 1, 3, 7, 4, 5]
i=6  ok[6]=2 → 先插 10         [6, 0, 8, 9, 1, 3, 7, 4, 5, 10, 2]
```

最终工序序列 $[6, 0, 8, 9, 1, 3, 7, 4, 5, 10, 2]$，写成工件号即 $[3, 1, 3, 4, 1, 2, 3, 2, 2, 4, 1]$（从 1 起标；同一工件的第 k 次出现即它的第 k 道工序）。直接按 (S, a) 排序则得 $[3, 3, 4, 4, 1, 3, 1, 2, 2, 2, 1]$——$J_3$、$J_4$ 的失败工序各自成块挤在最前，左移解码只能先串行做完它们，工件 J_1, J_2 被推到后面；**兜底把这两个块拆成 $[J_3, J_4, J_3, J_4]$ 的轮转序再均匀插回成功工序之间，各工件链就能并行推进。**

失败多于成功时（紧交期常见）：位点会变密、同一间隙连续插入多个失败工序，剩余的统一追加到末尾。例如 $n_{ok} = 2, n_{bad} = 8$ 时位点为 $0, 1, 1, 1, 1, 2, 2, 2$；$n_{ok} = 3, n_{bad} = 4$ 时位点为 $0, 1, 2, 3 (= n_{ok})$，第 4 个失败工序落到末尾。失败工序既不会被全部堆到最前（直接排序的问题），也不会被全部丢到最后。

（2b）完整流程走查（v2：LST 序 → 区间账本 → fail → OS 回写 → 解码）

算例：4 工件 / 3 机器 / 11 工序（`random_instance(4, 3, min_ops=2, max_ops=3, seed=0)`），d_i 由 `generate_due_dates(rng=default_rng(0))×0.8` 生成，即 $d = (200.47,\ 106.22,\ 55.92,\ 79.91)$；参数 rule=EDD、$\theta = 0$、$\gamma = 0$、`order="lst"`、`slack=0`（纯取最优，便于复现）。下表按 $\gamma = 0$ 记，选机即按 s 取最大；本例实跑 $\gamma = 1$ 时 S 与 OS 完全相同（γ 项只在少数候选上非零，且不足以翻转任何一步的 argmax），故两档可互换。

工件数据与 LST 工件序：

工件	工序（候选机 $m{:}p$）	$\mathrm{chain_min}_i$	d_i	LST_i
J_1	O_{11} M1:8；O_{12} M1:81；O_{13} M2:97 / M3:73	162	200.47	**38.47**
J_2	O_{21} M2:28 / M3:81；O_{22} M1:4 / M2:76 / M3:73；O_{23} M1:54 / M2:8 / M3:30	40	106.22	**66.22**
J_3	O_{31} M1:1 / M2:13；O_{32} M3:53；O_{33} M1:38 / M2:46	92	55.92	**−36.09**
J_4	O_{41} M1:95 / M2:65 / M3:84；O_{42} M1:58 / M2:72 / M3:84	123	79.91	**−43.09**

LST 降序为 $J_2 \to J_1 \to J_3 \to J_4$；$\bar{p} = 1222/23 \approx 53.13$，$\max_i h_i \bar{p}_{\mathrm{job},i} = 174$（$J_1$），故 $H = \max d_i + 174 = 374.47$。

逐工序走查（每行给"候选：$m{:}p$, $\lambda \to s$"；$\lambda = \min(LC, H) - (1+\theta)p$，$s$ 为区间账本给出的最晚可行开工；`–` 表示无空档）：

步	工序	工件	LC	候选	结果	S
1	O_{23}	J_2	106.22	M1:54, 52.22→52.22；**M2:8, 98.22→98.22**；M3:30, 76.22→76.22	M2	98.22
2	O_{22}	J_2	98.22	**M1:4, 94.22→94.22**；M2:76, 22.22→22.22；M3:73, 25.22→25.22	M1	94.22
3	O_{21}	J_2	94.22	**M2:28, 66.22→66.22**；M3:81, 13.22→13.22	M2	66.22
4	O_{13}	J_1	200.47	M2:97, 103.47→—；**M3:73, 127.47→127.47**	M3	127.47
5	O_{12}	J_1	127.47	**M1:81, 46.47→13.22**（避开 $[94.22, 98.22)$）	M1	13.22
6	O_{11}	J_1	13.22	**M1:8, 5.22→5.22**（避开 $[13.22, 94.22)$）	M1	5.22
7	O_{33}	J_3	55.92	M1:38, 17.91→—；**M2:46, 9.91→9.91**	M2	9.91
8	O_{32}	J_3	9.91	M3:53, −43.09→—	**fail**	0
9	O_{31}	J_3	0	M1:1, −1→—；M2:13, −13→—	**fail**	0
10	O_{42}	J_4	79.91	M1:58, 21.91→—；M2:72, 7.91→—；M3:84, −4.09→—	**fail**	0
11	O_{41}	J_4	0	M1:95、M2:65、M3:84 的 λ 均 < 0	**fail**	0

三个要点：

1. **区间账本能填空档**：第 4 步 O_{13} 在 M2 上放不下——M2 已被 $[66.22, 94.22)$、$[98.22, 106.22)$ 占据，$\lambda = 103.47$ 之前没有长 97 的空档（故 $s = \perp$）；M3 尚空，$s = \lambda = 127.47$。第 5、6 步则把 O_{12}, O_{11} 依次塞进 M1 的空档：O_{12} 落在 $[13.22, 94.22)$（紧贴 $[94.22, 98.22)$ 左侧），O_{11} 落

在 $[5.22, 13.22)$（紧贴 O_{12} 左侧）——标量账本（只能整体左移）做不到这种"填空"，这是 v1 fail 的主因。

2. **fail 不再毒化机器账本，但仍会沿工件链级联**：第 8 步 O_{32} 在 M3 上无空档（$\lambda = -43.09$）计 fail 后，$S = 0$ 只是贴边记录，**不写入 M3 的区间表**，M3 依旧空着；第 10 步 O_{42} 才是真实的空档冲突（M1/M2 的 λ 之前已被占满）；而第 9 步 O_{31}、第 11 步 O_{41} 的失败并非机器被占，而是 LC 被后继失败工序的 $S = 0$ 前传（$LC = S_{32} = 0$、$LC = S_{42} = 0$）导致 $\lambda < 0$——即 v2 的级联只沿**工件链（precedence）**发生，不再沿机器蔓延（v1 会把 $L[M3]$ 压到 0，使同机后续工序全部 fail）。本例 fail$= 4/11 \approx 0.36$，全部落在 J_3, J_4 这两个交期最紧、$LST < 0$ 的工件上。
3. **默认 `slack=1.0` 时第 1 步会出现容差池**：O_{23} 的 M2 得分 98.22、M3 得分 76.22，slack $\cdot\, \bar{p}_{O_{23}} = 1.0 \times 30.67 = 30.67$，$98.22 - 30.67 = 67.55$，故池为 $\{M2, M3\}$，按种子等概率抽一台——这是 TWTE 端多样性的来源（`slack=0` 则恒取 M2）。

OS 回写：成功序 $ok = [0, 8, 1, 3, 4, 5, 2]$（按 (S, a) 升序），失败组 J_3：$[6, 7]$、J_4：$[9, 10]$，工件轮转序 $bad_rr = [6, 9, 7, 10]$，位点 $[0, 2, 4, 6]$，合并得工序序列 $[6, 0, 8, 9, 1, 3, 7, 4, 5, 10, 2]$，写成工件号为 $[3, 1, 3, 4, 1, 2, 3, 2, 2, 4, 1]$（逐位细节见上文"改动 5 例子"）。同一 MS 下，左移解码：直接按 (S, a) 排序 $f_1 = 285$，$f_2 = 300.5$；兜底后 $f_1 = 163$，$f_2 = 199.0$。

（3）参数与实测

项	值	说明
`backward_order`	`"lst"`（默认）/ `"due_desc"` / `"edd"`	`"edd"` 为 v1 行为，供对照
`backward_slack`	1.0	选机容差（× 该工序 $\bar{p}$）；0 = 纯取最优
输出契约	不变	仍返回 `{ms, os, start, makespan, stats}`；`stats` 增 `fail`、`fail_rate`、`fail_ops`、`os_fallback`

`backward_slack` 消融（Mk1，池 100，`lst`；`segT` 为 30 个名额中来自反向构造的个数）：

slack	fail	`segT`	cov_{nd}	说明
0.0	0.28	4/30	0.14	纯 argmax，重复度太高
0.25	0.27	7/30	0.14	容差太窄
0.5	0.27	30/30	0.14	已满额但多样性不足
1.0（默认）	**0.30**	**30/30**	**0.29**	fail 略升，TWTE 端多样性翻倍
2.0	0.35	30/30	0.29	收益饱和、fail 上升

各算例（$T = 0.2$，`slack=1.0`，`lst`；9 组规则×θ×γ 各 10 次）：

算例	规模	$\max d_i / H$	v1 fail	v2 fail 均值	v2 最高	$segT$
Mk1	10 工件/55 工序/6 机	0.64	0.95	**0.292**	0.455	30/30
Mk3	15/150/8	0.66	0.96	**0.262**	0.353	30/30
Mk5	15/106/4	0.72	0.96	**0.465**	0.557	30/30
Mk10	20/240/15	0.62	0.97	**0.373**	0.467	30/30

交期放宽曲线（Mk1，$T = 0.1/0.2/0.5/1.0/2.0$）：v1 即使 $T = 2.0$ 仍有 0.82；v2 为 0.394→0.292→0.097→0.031→**0.000**。残差说明：$T = 0.2$ 下残余 ≈0.3 无法消除——Mk1 的确定性最小 makespan ≈ 54 大于最大交期 50.5，在"每个工件都必须按期完工"的反向口径下**至少一个工件必然落后**；$T \geq 1$ 后交期足够宽，fail 降到 0（上表末档）。Mk5 残差最高（4 台机器、$d_i/H = 0.72$ 最紧）。

DABI 池层（$N = 100$，seed=0，$T = 0.2$，`clip=0.95`）：B 族 Mk8 45 个种子的均值 $f_1 = 658,\ f_2 = 6074$（A 族 $853/9389$）；Mk12 $723/10127$（A $987/15905$）；Mk15 $682/9195$（A $835/10943$）；每个算例 100 个种子全部互异，cov_{nd} 不降。

> **副作用（须在论文中如实交代）**：兜底后失败占比高的 B 族不再"高 makespan"，反而在均值上优于 A；右上角（高 makespan + 高 TWTE）改由 A 族占据（Mk8/Mk12/Mk15 分别 33%/30%/30%，B 为 0%）。即"B 占据极端 TWTE 端"的图像消失，换来 B 解质量显著提升。若论文仍需"高 makespan + 零 TWTE"的极端端种子，可在 `segT` 中保留一部分**不做兜底**的构造（或调大 `backward_slack`），当前按方案一交付。

（4）实现与测试对应

- 代码：`dea_dd/algorithms/dabi.py` 的 `_MachineWindows`（区间表）、`_backward_job_order`（三种工件序）、`backward_construct(..., order, slack)`（改动 1–5）；
- 精度与可复现性：区间列表顺序确定，随机性只来自容差内等概率抽签，同种子同参数可复现；复杂度不变（Mk10 每机 ≤16 段）；
- 测试：`tests/test_dabi.py` 的 `TestBackwardRepair` 9 例（区间查询四分支、三种 order 复现与差异、区间不重叠、宽松交期 fail=0、fail 不毒化、兜底 OS 合法且严格优于直接排序、无失败不启用兜底），全量 98 例通过。

5.5 构造器 C：得分导向扰动

构造解是"规则化的"，容易同质。对候选池做一轮受控扰动：

- 父代从 A ∪ B 中随机取；
- 操作（1–3 步）：MS 把某工序换到 M_{ij} 内另一台机——**方向化**：makespan 端偏向更快的机器，TWTE 端偏向更慢/更闲的机器；OS 随机插入或交换一位；
- 每个候选都用双解码实测，保留"非支配于父代"或"拥挤距离更优"的，丢弃其余。

> 与文献里"纯随机 10%"相比，这里的扰动是**有方向、有父代、有筛选**的，每一份算力都花在已知好解的邻域里。

5.6 构造器 D：纯随机补充

`MS` 在 M_{ij} 内均匀取，`OS` 随机排列（每个工件恰出现 h_i 次）。作用有三：覆盖下限；充当网格覆盖（§5.8）里的"缺口格探针"；作为满编硬兜底（随机解空间近似无限，§5.8）。

5.7 编码回写

四个构造器都直接在调度序列上工作，最后统一回写成主编码（§3.1）：

字段	赋值	性质
`MS[a]`	构造选中的机器在 M_{ij} 内的下标 k	直接可用，无需修复
`OS`	**成功工序**按构造开工时刻 S **升序**列出；**失败工序**（$S = 0$ 仅贴边记录）不入直接排序，改按工件轮转序 + 按比例散布合并（§5.4.2 改动 5）	precedence 天然满足（前驱 S 更小；失败工序组内按工序序号升序）

5.8 配额与目标空间几何覆盖

P_0 的名额按方向切分（名额 = **段内互异解数**，不是构造调用次数，见下"在线去重"）：

段	占比	来源
makespan 端	30%	构造器 A，$\theta \in \Theta_F$，$\gamma = 0$，规则 SPT/LPT/AVPRO
TWTE 端	30%	构造器 B，$\theta \in \Theta_T$，$\gamma = 1$，规则 EDD/CR/S/RO
中间区	30%	A 与 B 交错：$\theta \in \{0, 0.5, 1\}$（取 $\Theta_F \cup \Theta_T$）、$\gamma \in \{0, 0.5\}$、规则 CR/AVPRO，共 $3 \times 2 \times 2 = 12$ 组
随机	10%	构造器 D

名额语义与在线去重（拒绝采样）：上表的"名额"指**段内互异解数**，不是构造调用次数。若先固定调用 30 次再去重，重复多的那段实际只剩 22 个左右，3:3:3:1 的方向比例会被随机稀释；在线去重让配额真正等于互异解数，并给出池子规模的硬保证（$|Cand| \geq N$）。四条规则缺一不可：

1. **全局唯一集合**：`Seen` 是整个 DABI 共用的 `MS+OS` 哈希集，不是每段一个。段内去重挡不住跨段撞车（如 A 段 θ=0.5/CR 与中间区某组合同解），跨段重复仍会让池子缩水；
2. **停滞上限**：同一 $(\theta, \text{rule}, \gamma)$ 组合的可产出互异解数有限（top-k 随机只在少数无并列的决策点上分支，多数步是确定性的），不许"凑够为止"。段级尝试预算 $\text{tries} \leq \rho \cdot \text{quota}$（$\rho = 3$），单组合另设约 8 倍份额的上限；任一触发即停，记 `Deficit[seg]`；
3. **轮转抽签**：同段内各**组合** $(\theta, \gamma, \text{rule})$（不是单独一个 θ）轮流各抽一次，不要"先把组合 A 凑满再换 B"。循环序**事先固定**：θ 升序 → rule 字典序 → γ 按段内档位升序（makespan 端 γ=0、TWTE 端 γ=1、中间区含 $0/0.5$ 两档）；同一实例同一 `Seed` 必须抽到同一串组合，**不可随机打乱**（否则同种子不可复现）。某组合产能提前耗尽时，它的份额自动转给同段其他组合，无需额外补偿逻辑；
4. **重试必须推进随机流**：`Seed++` 后再构造。三条随机流（`r_tie` / `r1` / `r2`，§5.3）都由种子派生，重试时若重置种子会重放同一个解，等于死循环。

例（makespan 端的轮转，$|\text{combos}| = 6$）：$\Theta_F = \{0,\ 0.5\}$、$\gamma = 0$、规则 $\{\text{SPT}, \text{LPT}, \text{AVPRO}\}$，按上述序展开为 `(0,SPT) → (0,LPT) → (0,AVPRO) → (0.5,SPT) → (0.5,LPT) → (0.5,AVPRO)`；配额 30、$\rho = 3$ 故尝试预算 90（15 轮）。`ptr` 每抽一次就前移，撞车不退格、只是不增加 `made`：

尝试	`ptr`	组合	θ	结果
1	1	(0, SPT)	0	收下，`made`=1
2	2	(0, LPT)	0	收下，`made`=2
3	3	(0, AVPRO)	0	撞车（`Seen` 已有）→ `made` 不变
4	4	(0.5, SPT)	0.5	收下，`made`=3
5	5	(0.5, LPT)	0.5	收下，`made`=4
6	6	(0.5, AVPRO)	0.5	收下，`made`=5
7	1	(0, SPT)	0	收下，`made`=6
8	2	(0, LPT)	0	收下，`made`=7

> 两个要点：①**每 6 次尝试里 $\theta=0$ 与 $\theta=0.5$ 各占 3 次**，但顺序是**块状**（先 3 个 0、再 3 个 0.5）而不是逐个交替；②上表到第 8 次为止 θ 实际出现 5:3，块状序在预算提前耗尽时会给先跑的一档更多机会——预算 90 远大于 6，正常无害；若要求严格无偏，用构造期独立 RNG 把组合表**打乱一次**后固定（不影响主搜索随机流，也不破坏同种子可复现）。

> - `Deficit` **是诊断量而非错误**：赤字率高说明该段的 $(\theta,\gamma,\text{rule})$ 组合区分力弱（与 §5.3 的"并列块大小"同类信号），可裁组合或换 θ；各组合的重复率（拒绝次数/尝试次数）同样可报。
> - **哈希只去决策空间**：不同 OS 排列经左移解码可能得到同一调度（同一组 (S,C)），`MS+OS` 哈希不同但 (f_1,f_2) 相同——这类解进 P_0 只占位。故解码后再做一道**目标空间量化键去重**：$\text{key}=\big(\lfloor f_1/\varepsilon\rceil,\lfloor f_2/\varepsilon\rceil\big)$，键相同只留一个代表。
> - **采样偏置**：越稀有的决策路径越难被重复、越容易通过去重，反复重抽会让该段种子向边角路径漂移。有上限时偏置有界，但要清楚在线去重得到的是"前若干**互异**解"而非"前若干**最优**解"。
> - **硬兜底**：构造器 D 的纯随机解空间近似无限，若各段都赤字导致 $|Cand|<N$，用 `while |Cand|<N: 收一个未重复的随机解` 补满——否则网格与拥挤距离都无法把 P_0 填到 N。

覆盖机制（按目标空间几何铺开，不依赖任何分解）：候选池全部双解码后，在归一化目标空间 $[0,1]^2$ 上划 $G\times G$ 网格：

$$\bar f_k(x)=\frac{f_k(x)-f_k^{min}}{f_k^{max}-f_k^{min}+\varepsilon},\qquad \text{cell}(x)=\Big(\lfloor G\bar f_1(x)\rfloor,\ \lfloor G\bar f_2(x)\rfloor\Big) \tag{13}$$

1. **每格保留 r 个代表**（$r\ge 1$）：第 1 个取该格内**离格中心最近**的解（几何覆盖，而非取最优）；其余按"格内 f_1 最小 → 格内 f_2 最小 → 格内最远点"的顺序补足，直到格内候选用完或到 r 个。双目标均极小化，故"格内 f_1 最小"与"格内 f_2 最小"正是该格自身微型前沿的两端，加中心恰好把一格撑成一小段；
2. **缺口格定向补种**：格内候选数 $<r$ 即为缺口格（$r=1$ 时退化为原来的"空格"），用该格对应的 $(\theta,\gamma,\text{rule})$ 组合补构造 $r-|c|$ 次，只收落进该格且未重复的解；构造是启发式的、补种可能落不进目标格，落空者按空格计入覆盖率，不无限重试；
3. **补满**：剩余名额 $N-\sum_c\min(|c|,r)$ 按**拥挤距离**从候选池补到 N，优先取非支配层，被支配解只作兜底；
4. **两级去重**：构造期按 `MS+OS` 哈希在线去重（上文"在线去重"四条规则），解码后按目标空间量化键再除一遍；下面网格内第 2、3 个代表最容易与中心代表撞车，故取代表时仍要查一次 `Seen`。

> 这样"两端覆盖"从"依赖多方向求解器收敛"变成**显式几何约束**，副产品是一个可报告的指标——**网格覆盖率**，可直接进消融表。
>
> **覆盖率的口径（两个必须同时报）**：①$\text{cov}=$ 非空格数$/G^2$；②$\text{cov}_{nd}=$（含非支配解的非空格数）$/(2G-1)$。原因：双目标均极小化时，一条单调前沿在 $G\times G$ 网格上呈阶梯状，**最多只穿过 $2G-1$ 个格**（$G=4$ 时仅 7 个），故 cov 的天然上界远小于 1、会随 G 变化；而 $(G-1,G-1)$ 这类"双差角"只有被支配解能填，只报 cov 会被平庸解刷分。再报一项 P_0 的非支配解占比，即可识别这种"灌水"。
>
> (G,r) **的定值依据**：
>
> - **供给**：网格只在双解码去重后的唯一解池（规模 $\approx N+q_P$）上切，需 $G^{2}r\lesssim0.5\,(N+q_P)$；否则大量缺口格补种落空，覆盖率低反映的是"格切太细"而非"解不好"。
> - **覆盖强度**：网格实际只决定 $G^{2}r$ 个名额，占比 $G^{2}r/N$；$N=100,G=4$ 时 $r=1$ 仅 16%、$r=2$ 为 32%、$r=3$ 为 48%——$r=1$ 时余下 84 个由拥挤距离决定、网格形同虚设，$r=3$ 则把近一半名额交给几何覆盖，所以 r 与 G 必须**成对**定、成对调。
> - **归一化敏感度**：式 (13) 的 f_k^{min}/f_k^{max} 取自池内极值，单个离群解会把其余解的 $\bar f$ 全压到 0 附近、整列格子被塞满；G 越大越敏感。实现上**默认对 f_k^{max} 取 95% 分位截断**（`params.grid_clip=0.95`，`None` 关闭）并在表注声明口径；否则单个离群解会让覆盖度量失真、消融不可比。

- **实现细节**：$\lfloor G\bar f\rfloor$ 在 $\bar f=1$ 处取到 G（越界），须钳到 $G-1$；"离格中心最近"一律在**归一化坐标**里算距离，避免 f_2（10^3 量级）压过 f_1（10^1 量级）。

建议初值：$N=100$ 取 $(G,r)=(4,3)$——48 个网格名额（占 48%）、52 个拥挤距离补满，平均每格 $(N+q_P)/16\approx8.1$ 个候选仍然充足（$G^{2}r=48\lesssim0.5\,(N+q_P)=65$）；且 $r=3$ 恰好把每格取成"中心 + 格内 f_1 最小 + 格内 f_2 最小"的**完整微型前沿**，$r=2$ 会丢掉 f_2 端那一份。消融分两条单因子线：定 $G=4$ 调 r 的 $\{(4,1),(4,2),(4,3)\}$，定 $r=3$ 调 G 的 $\{(3,3)\}$（$(5,3)$ 的 $G^{2}r=75>0.5(N+q_P)=65$，越过供给约束，不测），再加一组**同 $G^{2}r$ 的细网格对照** $(5,2)$（$50\approx$48），用来分离"名额多少"与"格级粗细"两个因素。

5.9 主伪代码

段名 `segF / segT / segM / segR` 即 §5.8 段表的四段：makespan 端 / TWTE 端 / 中间区 / 随机补充（构造器 D）；`quota(seg)` 是该段名额，`q` 为四段配额向量 (q_F,q_T,q_M,q_R)。扰动量另记为 q_P（§11），与段配额 q_M 区分。

```
DABI(实例, N, θ_F, θ_T, γ, q, G, r, ρ):
    p̄ ← mean(所有 p_ijk);  Cand ← ∅;  Seen ← ∅      // Seen: 全局 MS+OS 哈希
    Deficit ← ∅                                      // 各段缺额，诊断用
    // 配额：quota(segF)=quota(segT)=quota(segM)=0.3N，quota(segR)=0.1N
    // ---- 段 segF / segT / segM：在线去重（拒绝采样 + 轮转抽签） ----
    for seg in [segF, segT, segM]:
        combos ← seg 的组合表（segF 为 $2\times3=6$ 组、segT 为 $2\times3=6$ 组、segM 为 $3\times2\times2=12$ 组），按轮转序
        budget ← ρ·quota(seg);  tries ← made ← ptr ← 0
        while made < quota(seg) and tries < budget:
            (θ, γ, rule) ← combos[ptr mod |combos|];  ptr++
            x ← Construct(θ, γ, rule, Seed++)       // 重试必须推进随机流
            tries ← tries + 1
            if hash(x.MS, x.OS) ∉ Seen:
                Seen ← Seen ∪ {hash};  Cand ← Cand ∪ {x};  made ← made + 1
        Deficit[seg] ← quota(seg) − made
    // ---- C 得分导向扰动（同样过 Seen） ----
    for k in 1..q_P:
        x ← Perturb(random(Cand), 1..3 步)
        if hash(x.MS, x.OS) ∉ Seen:  Seen ← Seen ∪ {hash};  Cand ← Cand ∪ {x}
    // ---- D 随机补充 + 硬兜底（随机解空间近无限，保证满编） ----
    for k in 1..q_R:  收下一个未重复的 RandomChromosome()
    while |Cand| < N:  收下一个未重复的 RandomChromosome()
    // ---- 评测（名义加工时间，§9.2）：f1 左移、f2 右移（§3.2） ----
    for x in Cand:  (f1(x), f2(x)) ← 双解码(x)
    // ---- 二次去重：目标空间量化键（决策不同但调度相同的解） ----
    Cand ← 键去重(Cand, key = (⌊f1/ϵ⌋, ⌊f2/ϵ⌋))     // 去重后 |Cand| 可能略小于 N，由网格/拥挤补满回填
    // ---- 目标空间几何覆盖（式 13，每格取 r 个代表） ----
    S ← ∅
    for cell in G×G 网格:
        pick ← {该格中离格中心最近的解}                    // 第 1 个代表：几何代表
        pick ← pick ∪ {格内 f1 最小、格内 f2 最小、其余取最远点} 至 min(|cell|, r) 个
        if |pick| < r:  补种（按该格对应的 θ/γ/rule 再造 r−|pick| 次，只收落格且未重复者）
        S ← S ∪ pick
```

```
    S ← 拥挤距离补满(S, Cand, N)                    // 名额 N − Σ min(|cell|, r)，优先非支配层
    return S
```

5.10 复杂度与开销

- 单个构造：每道工序在 $|M_{ij}|$ 台机上打分，$O(O_{total}\cdot m)$；
- 双解码：右移传播按完工倒序一遍，$O(O_{total}\log O_{total})$；
- 候选池规模 $\approx N+q_P+G^2r$（10^2 量级；补种只在缺口格触发、最多 $G^2(r-1)$ 次构造）；含在线去重的 $\rho = 3$ 重抽预算后，构造调用 $\le 3N$、总开销 $10^5\sim10^6$ 次基本操作；
- 对比"随机起点 + 迭代收敛"的预热链路（约 5×10^3 次完整解码）**低约一个数量级（≈17 倍）**，且全部为确定性构造，无收敛过程、无超参数曲线。

5.11 主要参数

参数与建议初值统一列在 §11 参数表（$\Theta_F,\Theta_T,\gamma,q,G,r,\rho,k$）。三个调参要点：

1. Θ,γ 由消融确定（§12 第 5 条**第三组补充消融**：只跑 DABI、固定 (G, r) 与配额，比较两端有解率与 P_0 覆盖），不靠经验；
2. top-k 随机（$k = 2$）是"方向内多样性"的主要来源，$k=1$ 会让同一 (θ,rule) 只产出一个解；
3. 配额 $3:3:3:1$ 与 $(G,r)=(4,3)$ 在 Mk 系列上不敏感，先固定；(G, r) 须**成对**调、成对报（G^2r 才是"网格实际管住多少名额"），且一旦偏离 $(4,3)$ 就要在表注声明覆盖率口径（cov 的分母随 G 变，见 §5.8）。

5.12 验证设计

见 §12 的指标（第 2 条）、对比算法（第 3 条）与消融（第 5 条），此处只固定判据：

- **基线四组**：纯随机初始化；混合规则初始化（6 规则 + 随机）；GS/LS 混合（routing 贪心 6:3:1）；离散 LHS 网格。
- **指标**：P_0 的 $f_1^{best},f_2^{best},\bar f_1,\bar f_2$；归一化 HV；网格覆盖率（$\mathrm{cov}$ 与 cov_{nd} 两口径，§5.8）+ P_0 非支配解占比；各段赤字率（`Deficit`/配额）与各 (θ,rule) 组合重复率（规则区分力诊断，§5.8）；两端"有解率"；**最终**——从 P_0 出发跑完整 DBC-NSGA-II 后的 HV（最重要的一个）。
- **消融**：θ 网格、γ、反向构造占比、扰动占比、(G, r) 成对（$G = 4$ 时 $\{(4,1),(4,2),(4,3)\}$；$r=3$ 时 $\{(3,3)\}$；细网格对照 $(5,2)$）、回写方式（按 S 升序 vs 位置编码；反向失败工序另做"兜底前/后"对照，见 §5.4.2）。

5.13 风险与兜底

风险	诊断	兜底
交期过松 → TWTE 端退化（全部 $f_2=0$，两目标退化为单目标）	随机种群里 $f_2=0$ 占比 >90%	收紧交期紧度（§12.1 的 T），使随机解下 40–60% 工件拖期
紧交期 → 反向构造失败率过高	`fail` 率长期 >0.5（v2 已内建区间账本 + LST 序，见 §5.4.2）	先降 θ（$1\to0.5\to0$，实测 fail 约降 0.03），再换工件序（`lst` → `due_desc`，差异很小）；`backward_slack` 主要用于多样性、不是降 fail 的手段（$0\to1.0$ 时 fail 反从 0.28 升到 0.30）；仍不改善才把 `segT` 换成"前向构造 + 右移解码"生成 TWTE 端种子（方案 1）；残余 ≈0.3 属交期地板，如实报告即可
失败工序成块排在最前 → 解落右上角	同一 MS 下兜底前后 f_1 差 2–3 倍	失败工序按工件轮转序 + 按比例散布（§5.4.2 改动 5），`os_fallback` 标记
γ 过大拖坏 makespan	TWTE 组解 f_1 比 makespan 组差 >2 倍	把 TWTE 组机器池限制为"当前负载低于中位数"的子集
构造解同质、去重后不足 N	网格覆盖率低（cov 与 P_0 非支配占比同时低）、各段赤字率高	先降 (G, r)（格切太细时补种必落空），再提高 k 到 3、加大扰动比例、放宽配额；最终由纯随机硬兜底 `while |Cand|<N` 保证满编（§5.8/§5.9）

5.14 与已有初始化方法的对应

来源	借用	本方案的改造
混合规则初始化（NSGA-II + 仿真启发式）	逐步调度构造 + 多条调度规则 + 反向编码回写	规则按方向分组；补反向构造；按实际开工时刻 S 升序回写 OS（失败工序见 §5.4.2 改动 5）
GS/LS 初始化	账本式路由贪心（$T+p$）	扩展为 θ 参数族 + 解耦账本 $\gamma\varphi$；不再只做路由段
区间/时间窗调度（JIT 倒排、滚动时域）	每台机器维护不重叠占位区间、按"最晚可行开工"落位	与 §5.2 账本路由合并：θ 控制提前裕度、$\gamma\varphi$ 只做偏好排序（§5.4.2 改动 2）
对角线性均匀初始化（DLU）	确定性覆盖 + 用多样性指标评价初始化	从"连续空间等距取点"改为**目标空间网格覆盖**（离散版）
MOEA/D 的分方向初始化思想	按方向分配种子	方向配额直接作用于目标空间几何（§5.8）

5.15 备选：离散短程局部搜索

若希望在构造之上再加一层精修，可对 P_0 中目标空间靠前的约 30% 解做**离散短程局部搜索**（`可尝试的方法.md` 方案 B）：

- 邻域：MS 换机/交换、OS swap/insert/inverse；
- 接受准则：**Pareto 支配改进**（$x' 支配 x，或在归一化空间里拥挤距离更优）则接受；
- 步数：约 30 步（避免收敛死、保持端内多样）。

该精修**纯离散、不引入实值编码**，与阶段二的离散邻域算子同源；创新① 的可辩护性不依赖它，它只作为可选增强，在消融中作为"有/无精修"一组对比。

---

6. 阶段二：主搜索（改进 NSGA-II）

6.1 交叉

- **OS 段**：POX（Precedence Operation Crossover，工序优先级交叉）。
  1. 把工件集随机分为 J_1, J_2 两组；
  2. 子代 1 保留父代 1 中属于 J_1 的工件位置不变，空位用父代 2 中属于 J_2 的工件按原顺序填入；子代 2 对称生成。
- **MS 段**：UX（均匀交叉）。逐工序以概率 p_c 从父代 1/2 之中取对应基因。

6.1.1 交叉示例（延续 §3.1.1 算例）

OS 段 POX：设父代 $P_1=OS=[1,2,1,3,2]$、$P_2=[2,1,3,1,2]$，工件集随机分成 $J_1=\{1,3\}$、$J_2=\{2\}$：

- 子代 1：保留 P_1 中属于 J_1（工件 1、3）的位置 → $[1,_,1,3,_]$；空位按 P_2 中属于 J_2（工件 2）的出现顺序填入 → $[1,2,1,3,2]$；
- 子代 2：对称。保留 P_2 中属于 J_1 的位置 → $[_,1,3,1,_]$；空位按 P_1 中工件 2 的出现顺序填入 → $[2,1,3,1,2]$。

MS 段 UX：设两父代的 MS 分别为 $P_1=[0,1,0,0,0]$、$P_2=[1,0,1,1,0]$，取 $p_c=0.5$，逐工序掷随机数 $r_a\sim U(0,1)$：

工序下标 a	O_{11}	O_{12}	O_{21}	O_{22}	O_{31}
P_1 基因为	0	1	0	0	0
P_2 基因为	1	0	1	1	0
r_a（示例）	0.3	0.7	0.4	0.9	0.55
$r_a<p_c$?	是→取 P_1	否→取 P_2	是→取 P_1	否→取 P_2	否→取 P_2
子代 1 MS	**0**	**0**	**0**	**1**	**0**

子代 2 在每一位取"另一方"的基因（互补），得 $[1,1,1,0,0]$。

> **可行性说明**：MS 每个基因独立对应一道工序在其**可选机器集 M_{ij} 内**的机器下标，UX 只是在同一道工序自己的 M_{ij} 范围内二选一，因此**交叉不会产生非法机器**、无需修复。这是 MS 段采用 UX 的主要原因之一（换成单点/两点交叉同样保可行，但 UX 混合更均匀、破坏性更小，与 POX 配合沿用 FJSP 主流做法）。

6.2 变异（含创新⑤：动态变异位数）

- **OS 变异**：随机选若干个位置交换（swap）、插入（insert）或逆序（inverse）；
- **MS 变异**：随机选若干个工序，替换为其可选机器集 M_{ij} 内另一台机器；
- **创新⑤——变异位数随规模与迭代动态决定**：不再固定"变两位"，改为"**规模缩放 × 迭代衰减**"双层自适应：
- **变异概率 p_m**：变异不是对每个子代都执行，而是**以概率 p_m 决定是否变异**——命中则对该子代的 OS 与 MS 各改 K_{mut} 个位置，未命中则跳过变异、直接保留交叉结果。p_m 与交叉概率 p_c 解耦（p_c 管"是否交叉"、p_m 管"是否变异"），缺省 $p_m=0.2$（与实现 `Params.p_m` 默认值一致）；与创新⑤的分工是：$\delta(gen)$ 管**改几个位置**，p_m 管**改不改**。

$$K_{mut}(gen) = \max\Big(2,\ \lceil \delta(gen)\cdot O_{total} \rceil\Big), \qquad \delta(gen) = \delta_{max}\cdot\Big(\frac{\delta_{min}}{\delta_{max}}\Big)^{\frac{gen}{gen_{max}}} \tag{14}$$

其中 $O_{total} = \sum_i h_i$（大实例多改几位、小实例少改几位），$\delta(gen)$ 随迭代从 $\delta_{max}=0.05$ 指数衰减到 $\delta_{min}=0.01$（**前中期大变异探索、后期小变异精修**），下界 $\max(2,\cdot)$ 保证终局至少改 2 位、避免完全停滞。两个维度正交：规模维管"改多少比例"，迭代维管"何时多改、何时少改"，相较只按规模固定的 δ 更利于后期收敛。

6.3 非支配排序 + 拥挤距离 + 精英保留

1. 父代 P（规模 N）经交叉变异产生子代 Q；
2. 合并 $R=P\cup Q$，做快速非支配排序分前沿 $F_1,F_2,\dots$；
3. **拥挤距离用标准 NSGA-II 公式**（每目标邻差除以该目标跨度 $f^{max}-f^{min}$，即两端跨度；该除法本身即尺度归一化，见下）；
4. 按前沿等级 + 拥挤距离截断到规模 N，精英保留。

> **第 3 点含义（回应"是否要先归一化"）**：标准 NSGA-II 拥挤距离本就自带"除以两端跨度"（对目标 m：相邻插值 $f_{m,i+1}-f_{m,i-1}$ 再除以该目标跨度 $f_m^{max}-f_m^{min}$），这个"除以跨度"就是尺度对齐，**并无"先归一化再算"的额外步骤。**本设计只需约定一件事：这个跨度 $f_m^{max}-f_m^{min}$ 用**§4.1 的全局逐代极值**（式 (8)-(10)），而非每个前沿自己的端点极

> 值。原因见 §6.3.1 示例——局部跨度会因前沿在某目标"已收敛"或"端点离群"而失真，全局跨度单调稳定、不放大窄目标。

6.3.1 为什么跨度要用全局逐代极值（示例）

一个前沿 F_k 若在某目标上**已经收敛（跨度很窄）**，局部跨度会把自己除得很小，从而把该目标上本可忽略的微小差异**放大到主导拥挤距离**，反过来盖过另一目标上真实的分布差异，导致截断"淘汰错点"。

设 F_k 有 5 个互不支配的解（makespan 按升序时 TWTE 严格递减，f_1 已收敛到 ~100，f_2 尚未收敛 $0\sim1000$）：

标号	f_1 (makespan)	f_2 (TWTE)	说明
A	100.0	1000	左端点
X	100.1	100	f_2 方向的孤岛（$1000\to100$ 大缺口），必须保留
Y	100.6	95	在 f_2 上紧贴 X（只差 5），冗余
Z	100.7	50	分布相对均衡
E	101.0	0	右端点

要淘汰 1 个，中间点 X,Y,Z 的拥挤距离如下（A,E 端点记 $+\infty$）：

- **局部跨度**：$\Delta_1^{loc}=101.0-100.0=1.0$，$\Delta_2^{loc}=1000-0=1000$；
- **全局逐代跨度**（当前全种群，示例值）：$\Delta_1^{glob}=420$（$f_1\in[80,500]$），$\Delta_2^{glob}=2000$（$f_2\in[0,2000]$）。

点	分子 $(\Delta f_1,\Delta f_2)$	局部极值拥挤距离	全局极值拥挤距离
X	$(0.6,\ 905)$	$0.6/1.0+905/1000=1.505$	$0.6/420+905/2000=0.454$
Y	$(0.6,\ 50)$	$0.6/1.0+50/1000=0.650$	$0.6/420+50/2000=0.026$
Z	$(0.4,\ 95)$	$0.4/1.0+95/1000=0.495$	$0.4/420+95/2000=0.048$

两种排序结果截然不同：

- **局部极值**排序 $X(1.505)>Y(0.650)>Z(0.495)$ → 淘汰 Z ✗；
- **全局极值**排序 $X(0.454)>Z(0.048)>Y(0.026)$ → 淘汰 Y ✓。

正确答案是**淘汰 Y**：它在 f_2 上几乎贴着 X（只差 5），是真正的冗余点；X 是 f_2 方向的孤岛、Z 分布更均衡，都应保留。

为什么会错：局部方案里 f_1 的权重是 $1/\Delta_1^{loc}=1/1.0=1$、f_2 是 $1/\Delta_2^{loc}=1/1000=0.001$，**两者相差 1000 倍**——因为 f_1 在前沿内已收敛、跨度只有 1.0，于是 f_1 上 0.6 的微小差被拉成 0.6，盖过了 f_2 上 X 的真实孤岛（905）与 Y 的冗余贴邻（50）之分。全局方案用稳定跨度，权重比 $1/420:1/2000\approx 4.8$，f_2 的真实分布不再被 f_1 的收敛噪声掩盖，故能正确淘汰 Y。

引申（端点离群同样失效）：局部跨度 = 端点差，对**端点离群值**敏感——前沿里混进一个 f_1 离群点就把全体拥挤距离统一缩水；全局逐代极值单调只增不减、由整个种群累计得到，对单点离群不敏感、代际稳定，且与拥挤距离共用同一套归一化坐标（式 (8)(9)）。

---

7. 交货期导向的双向邻域（创新③）

7.1 设计思想（对应结论 A/B）

> **设计原则（总纲）**：凡声称改善 f_2（TWTE）的算子，必须通过**改动 MS/OS 编码**来改变调度结构；只动时间轴、不改变编码的操作（如凭空插 idle）不属于搜索算子，而是**解码器的职责**（§3.2b）。这是区分"搜索层（改编码）"与"评估层（译码算值）"的一条硬规则，R1 的换慢机/顺序后移即据此设计。

传统 FJSP 邻域（交换/插入工序、换快机）本质都在"左移"意义上改善 makespan；而 TWTE 需要"右移对齐交货期"。因此设计**两类方向互补**的邻域算子：

方向	算子	作用目标
左移（压缩）	L1 关键工序换更快机器 / 提前插入；L2 同机两工序交换；L3 插入更早空闲间隙	缩小 f_1
右移（对齐）	R1 换慢机 / 顺序后移，把早到工件右移对齐 d_i（改 MS/OS 编码，非插 idle）	消除早到惩罚，改善 f_2

7.2 左移类算子 L1–L3

- **L1（关键路径换机）**：定位于关键路径（支配 makespan 的最长链）上的工序，将其分配改为可选机器集内加工最快的机器（若更短则接受）。
- **L2（同机交换）**：在瓶颈机器上随机交换两道相邻工序的顺序。
- **L3（前向插入）**：把某工序插入到其机器上更早的空闲间隙（不违反 precedence 与前序）。

7.3 右移类算子 R1（编码层右移：换慢机 / 顺序后移）

关键前提（回应对 R1 有效性的质疑）：MS+OS 编码只表达"机器选择 + 工序顺序"，**不包含 idle（空闲）**；idle 是解码器在给定编码下**确定性投影**出来的结果，不在搜索空间内。因此"直接在某工序前插 idle"不改编码——候选解与父解的 MS/OS 完全相同，评估时右移解码（§3.2b）又把它重算成同一结果，**插 idle 无效、也不可被遗传保留。**R1 要真正起作用，必须**通过改动编码**把"推迟完工、对齐交货期"的结构写进 MS/OS。具体两个编码层右移动作：

- **R1a 换慢机（machine slow-down，改 MS）**：对早到工件 i（$E_i>0$）的某道工序，把机器换成其可选机器集 M_{ij} 内**加工时间更长**的机器，延长该工序占用，使 C_i 推迟、凑近 d_i，消除早到；
- **R1b 顺序后移（postpone，改 OS）**：在早到工件某道工序所在的机器上，把它与机器序列中**靠后**的工序交换（或整体移到更靠后的位置），推迟开工，使 C_i 逼近 d_i（不违反 precedence）。

共同接受条件（保证 TWTE 单调不增）：改动后**不使任何工件的 T_i 增加**（不新增/不扩大拖期），且不破坏工序先后约束。

效果：E_i 下降、f_2 下降；f_1 允许不变或略升（换慢机额外占用机器时间、顺序后移推迟原工序，该算子在 Pareto 上向 TWTE 端移动）。

> **与右移解码的分工**：右移解码（§3.2b）负责**在固定编码下**把 f_2 用"最优右移（idle 自动插入）"算出来；R1 负责**通过改 MS/OS** 把"右移对齐交货期"的结构固化进编码、供后续遗传与搜索。一个是"算值时被动右移"（评估层），一个是"改编码主动右移"（搜索层），不重复。

7.4 邻域评估

每个邻域算子生成的候选解都用**双解码**评估 (f_1, f_2)，按 §8.1 的口径判定成功：**支配父代，或在归一化空间里拥挤距离更优**则用候选解替换父代，否则丢弃；成功率统计与 §8 的自适应选择联动。

---

8. 自适应算子选择 + 延迟触发（创新④）

8.1 成功率统计

对每个算子 $o\in\{L1,L2,L3,R1\}$，维护最近 $L=10$ 次执行的成功数 S_o 与失败数 F_o（"成功"=产生 Pareto 非支配改进：支配父代，或在归一化空间里拥挤距离更优）。

8.2 轮盘赌选择

算子 o 被选概率：

$$p_o = \frac{S_o/(S_o+F_o)+\mu}{\sum_{o'}\left(S_{o'}/(S_{o'}+F_{o'})+\mu\right)}, \qquad \mu = 0.05\ (\text{小基线防零概率}) \tag{15}$$

（分母按 $S_{o'}/(S_{o'}+F_{o'})$ 归一，与分子一致；先前公式里多余的 $/F_{o'}$ 是笔误，已改正。）成功率越高的算子越常被调用，随进化状态自适应。

冷启动（$S_o=F_o=0$，所有算子尚无执行记录）：此时 $S_o/(S_o+F_o)=0/0$ 无定义。处理约定——当 $S_o+F_o=0$ 时把成功率项记作 0，只用基线 μ，于是

$$p_o = \frac{0+\mu}{\sum_{o'}(0+\mu)} = \frac{\mu}{|\mathcal{O}|\cdot\mu} = \frac{1}{|\mathcal{O}|} \tag{15a}$$

其中 $\mathcal{O}=\{L1,L2,L3,R1\}$，$|\mathcal{O}|=4$。即**初期四个算子等概率（各 0.25）**，随统计窗口 $L=10$ 内样本累积，成功率项逐渐主导、分布自适应分化；$\mu=0.05$ 保证任何算子即使连续失败也不至于概率归零、仍有 $\mu/\sum(\cdot)>0$ 的被试探机会。

8.3 延迟触发

局部搜索**推迟到迭代后期**才启动：

$$\text{触发条件：}\quad gen \geq 0.8\cdot gen_{max} \tag{16}$$

理由（沿用 DLSEA）：前期以全局探索为主、不浪费算力局部精修；后期集中精化前沿质量。

---

9. 随机加工时间与两阶段评估口径（simheuristic）

9.1 随机加工时间模型（对数正态，参考 Nessari 2024）

本方案沿用 [Nessari 2024] 的对数正态加工时间模型（其式 11–14）。工序 O_{ij} 在机器 M_k 上的加工时间 $\tilde p_{ijk}$ 服从**对数正态分布**，且其**期望等于名义值** p_{ijk}（即前文确定性模型使用的加工时间）：

$$\mathbb{E}[\tilde p_{ijk}] = p_{ijk}, \qquad \mathrm{Var}[\tilde p_{ijk}] = UL\cdot p_{ijk}, \qquad UL\in\{0.1,\ 0.25,\ 0.5\} \tag{17}$$

其中 UL 为不确定性水平（uncertainty level），三档从小到大对应低/中/高波动。对数正态由 (μ_{ijk},σ_{ijk}) 刻画（即 $\ln\tilde p_{ijk}\sim\mathcal N(\mu_{ijk},\sigma_{ijk}^2)$），用期望—方差矩匹配反解：

$$\mu_{ijk} = \ln p_{ijk} - \frac{1}{2}\ln\!\left(1+\frac{\mathrm{Var}[\tilde p_{ijk}]}{p_{ijk}^2}\right), \qquad \sigma_{ijk} = \sqrt{\ln\!\left(1+\frac{\mathrm{Var}[\tilde p_{ijk}]}{p_{ijk}^2}\right)} \tag{17a}$$

代入式 (17) 得 $\sigma_{ijk}=\sqrt{\ln(1+UL/p_{ijk})}$、$\mu_{ijk}=\ln p_{ijk}-\tfrac12\sigma_{ijk}^2$。对数正态天然保证 $\tilde p_{ijk}>0$，无需截断。于是 makespan 与 TWTE 也随机化，记为 $f_1(\tilde p),\ f_2(\tilde p)$；优化目标取**期望** $\mathbb{E}[f_1],\ \mathbb{E}[f_2]$（必要时可叠加分位数/CVaR 表达鲁棒性，本方案先以期望为主，稳健性 gap Δ_o 保留为可选补充指标）。

9.2 两阶段评估口径（用户方案：阶段一固定、阶段二随机）

- **阶段一（构造初始化）= 固定（名义）加工时间**：DABI 直接用 p_{ijk} 做构造，再用一次确定性左移/右移解码得 (f_1, f_2)；若启用可选精修（§5.15），其比较也在该口径下计算。评估便宜，目的仅是**快速造出结构多样、覆盖两端的初始种群**，不追求随机口径下的精确值。
- **阶段二（NSGA-II 主搜索）= 随机加工时间 + 蒙特卡洛评估**：对每个解 x 采样 R 个独立场景 $\{\tilde p^{(r)}\}_{r=1}^{R}$，各解码一次得 $(f_1^{(r)},f_2^{(r)})$，取期望：

$$\hat f_1(x) = \frac{1}{R}\sum_{r=1}^{R} f_1\big(\tilde p^{(r)}\big), \qquad \hat f_2(x) = \frac{1}{R}\sum_{r=1}^{R} f_2\big(\tilde p^{(r)}\big) \tag{18}$$

此后支配关系、非支配排序、拥挤距离、min-max 归一化**全部作用在 $(\hat f_1,\hat f_2)$ 上。**

> **可行性结论**：可行，且正是 [Nessari 2024] "确定性预热 → 随机仿真精化"的 simheuristic 范式。阶段一固定口径只做**结构初始化**、尺度不精确无碍；阶段二切换随机口径做真正优化，最终前沿即随机鲁棒前沿。代价是阶段二评估成本放大 R 倍，靠 §9.4 的采样分层缓解。

9.3 归一化基座的衔接（唯一必须处理的技术点）

阶段一在**固定口径**下得到的 f^{min}/f^{max} 与阶段二**随机口径**的 $\hat f^{min}/\hat f^{max}$ 尺度不同（随机右尾通常抬高期望成本）。处理：**进入阶段二时，用随机评估对初始种群 P_0 重新建立 min/max 基座**（仍沿用 §4.1 逐代单调更新机制，只是基座改由 P_0 的随机口径值重新起点），随后随迭代继续单调更新。找不到这个尺度衔接点会导致归一化失真。

9.4 蒙特卡洛采样规模（短仿真 vs 长仿真）

- **迭代中（短仿真/short simulation）**：对一切解的期望评估统一取 $R=20$（粗评，足够支配比较与排序；对应 §10 伪代码中的"评估 (MC, 式18)"与局部精修评估）；
- **最终前沿（长仿真/long simulation）**：算法收敛后，对非支配前沿 P^* 的每个解做 $R=10000$ 次蒙特卡洛，得高精度期望 $(\hat f_1,\hat f_2)$ 与稳健性统计；
- **公共随机数（CRN）**：比较不同解时复用同一组场景 $\{\tilde p^{(r)}\}$，降低比较方差，使支配判断更稳定。

9.5 输出

取 $(\hat f_1,\hat f_2)$（经 $R=10000$ 的长仿真）下的**非支配前沿** P^* 作为最终解集；对每个前沿解报告期望值，可选加报标准差/分位数与稳健性 gap $\Delta_o=(f_s-f_d)/f_d$（f_s 随机仿真值、f_d 固定口径值）。

---

10. 算法总伪代码（DBC-NSGA-II）

```
输入: n,m,{h_i},{M_ij},{p_ijk},{d_i},UL; 参数
N,Θ_F,Θ_T,γ,q,G,r,ρ,gen_max,p_c,p_m,δ,R
输出: 非支配前沿 P*

// ---- 阶段一：交货期感知双向构造初始化（固定加工时间, §5/§9.2） ----
P = DABI(实例, N, Θ_F, Θ_T, γ, q, G, r, ρ)                  // 前向+反向构造、扰动、随机、网格覆盖
// 可选：对 P 中目标空间靠前的 ~30% 解做 30 步离散短程精修（§5.15）

// ---- 阶段二：改进 NSGA-II 主搜索（随机加工时间, §9） ----
评估 P（蒙特卡洛, 式18）；重建 min/max 基座(§9.3)
for gen in 1..gen_max:
```

```
    Q = crossover(P1,P2):  OS=POX, MS=UX    // 父代选择：锦标赛选 p1 + 随机 p2，共 N/2 对（§6.2；p1=p2 允许）
    Q = mutation(Q):      以概率 p_m 变异; 动态变异位数(式14), OS swap/insert/inverse, MS 换机
    评估 Q（蒙特卡洛, 式18）; 更新 min/max(逐代, 式8)
    if gen ≥ 0.8·gen_max:                          // 延迟触发, 创新④
        for x in Q (部分样本):                     // 对子代局部精修
            o = 轮盘赌选择算子(式15)               // 自适应, 创新④
            x' = apply_o(x)                        // L1/L2/L3/R1, 创新③
            评估 x'; 若支配 x 则用 x' 替换 Q 中的 x
    R = P ∪ Q
    非支配排序 → 前沿 F1,F2,...
    拥挤距离(在 min-max 归一化空间, 式9)
    P = 截断到 N (精英保留)
P* = non_dominated(P)
对 P* 每个解做长仿真（R=10000, §9.4）; 重算 (\hat f_1,\hat f_2) 与稳健性统计
return P*
```

> **顺序说明（回应"先局部精修还是先排序截断"）**：局部搜索放在**非支配排序/截断之前**、作用在**子代 Q** 上。理由：memetic NSGA-II 的标准做法是让局部精修出的个体**参与本代的环境选择**（排序 + 拥挤距离 + 截断），而不是在截断之后才直接替换幸存者——后者会绕过拥挤距离、且被替换者虽已被"选为精英"却未与整池重新排序。放到 Q 上还避免了对父代精英反复扰动、保护了已收敛的 makespan 端。

---

11. 参数设置（建议初值）

参数	建议值	说明
种群规模 N	100	与 DLSEA/NSGA-II 可比
端路由参数 Θ_F	{0, 0.5}	makespan 端选机打分（式 10）
端路由参数 Θ_T	{0.5, 1}	TWTE 端选机打分（式 10）
解耦强度 γ	0 / 0.5 / 1	makespan 端 / 中间区 / TWTE 端
top-k 随机	$k = 2$	破平局，产生方向内变体
反向工件序 `backward_order`	`"lst"`（默认；`"due_desc"` / `"edd"` 供消融）	$LST_i = d_i - \text{chain_min}_i$ 降序（§5.4.2 改动 1）
反向选机容差 `backward_slack`	1.0	可行机器中与最优得分差 $\le \text{slack} \cdot \bar{p}_{ij}$ 的等概率抽签；0 = 纯 $\arg\max$，fail 与 0.25/0.5 基本相同（0.28 vs 0.27）但 `segT` 只 4/30（§5.4.2）
种子配额 $q_F:q_T:q_M:q_R$	3:3:3:1	四段比例（§5.8）
覆盖网格 G	4	目标空间网格分辨率（式 13）；须与 r 成对定
格内代表数 r	3	每格保留的代表数；约束 $G^2r\approx0.3\!\sim\!0.5\,N$ 且 $G^2r\lesssim0.5\,(N{+}q_P)$（§5.8）
网格归一化截断 `grid_clip`	0.95	网格归一化时对 f_k^{max} 取 95% 分位（§5.8，抗离群解）；`None` 关闭
重抽预算系数 ρ	3	在线去重的段级构造尝试上限 $=\rho\times$ 段配额；单组合另设约 8 倍份额上限（§5.8）
扰动量 q_P	约 0.3N	得分导向扰动的**条数**，每条 1–3 步；与段配额 q_M 区分（§5.8 段表）
交叉概率 p_c	0.8~0.9	POX/UX
变异比例 δ	0.01~0.05	动态位数（式14），$\delta_{max}=0.05$、$\delta_{min}=0.01$
变异概率 p_m	0.2	每个子代是否执行变异（OS+MS 一起）；实现取 0.2，§6.2 同
主搜索代数 gen_{max}	300~500	或按 Fes 上限
LS 触发点	$0.8\cdot gen_{max}$	延迟触发
LS 窗口 L	10	成功率统计窗口
$\varepsilon,\ \mu$	$10^{-6},\ 0.05$	除零保护 / 算子基线
不确定性水平 UL	{0.1, 0.25, 0.5}	对数正态方差档位（式17）
短仿真 R	20	迭代期 MC 场景数（§9.4）
长仿真 R^*	10000	最终前沿 MC 场景数（§9.4）

> 所有参数在后续实验阶段用**试验设计/消融**微调；对比算法须保持种群规模、总评价次数一致以保证公平。

---

12. 实验验证思路（后续落地）

1. **算例**：先用 **Brandimarte Mk01–Mk10**（10 例，10×6 → 20×15，机器相关时间的经典 FJSP，契合 §1）验证效果；效果确认后再推广到 **Hurink** 四档（s/e/r/v，同一 base 实例 × 4 种柔性）检验柔性敏感性。数据取 `Lei-Kun/FJSP-benchmarks`（`.fjs` 格式，组织方式见 §12.1）；**交货期按**

Bagheri & Zandieh (2011) 法生成（公式见 §12.1）。随机口径按 §9 对数正态、取 $UL\in\{0.1,0.25,0.5\}$ 三档分别实验。
2. **评价指标**：HV（超体积）、IGD、C-metric（集覆盖率）、NPS（非支配解数），并单独报告 makespan 端与 TWTE 端的极值 gap。
3. **对比算法**：NSGA-II、MOEA/D、DLSEA、Sim-NSGA-II。
4. **统计**：30 次独立运行，Wilcoxon 符号秩检验（α=0.05）。
5. **消融**：逐个关掉创新①（退化为纯随机初始化）、③（去掉 R1 右移类算子）、④（固定算子概率/固定触发点）、⑤（固定变异 2 位），验证各自独立增益。另设三组补充消融：有/无 DABI 各构造器、有/无 §5.15 短程精修、**阶段一参数消融**（只跑 DABI、不跑进化：Θ_F 取 $\{0\}$ 与 $\{0,0.5\}$ 对比，Θ_T 取 $\{1\}$ 与 $\{0.5,1\}$ 对比，γ 取 $0/0.5/1$ 三档；每组 5 个种子，比较 P_0 的"两端有解率 / f_2^{best} / 归一化 HV / cov_{nd}"，结论回写 §11 参数表与 §5.11 第 1 点）。**重点展示"不塌陷的完整 Pareto 前沿"**（TWTE 端有解、无右移空缺）。

12.1 基准算例的数据组织（Brandimarte 与 Hurink）

有两种数据集：[Lei-Kun/FJSP-benchmarks: The public benchmark instances of flexible job shop scheduling problem](https://github.com/Lei-Kun/FJSP-benchmarks)（优先使用这个）、https://github.com/cls1277/FJSP-benchmark

两套基准均采用同一种 `.fjs` 纯文本格式，三要素：首行 `工件数 J  机器数 M  平均每工序候选机器数`；之后每工件一行 `工序数 n_j  [每工序：候选机器数 k (机器号 加工时间) × k 组]`。

Brandimarte (1993)（原生设计的 FJSP 基准，Mk01–Mk10 共 10 个，规模 10×6 → 20×15）：

```
10  6  2                          ← 10 工件，6 机器，平均每工序 2 台候选
 6  2 1 5 3 4   ...               ← 工件1 有 6 道工序；工序1 = 2 台候选：
                                     (机器1，用时5)、(机器3，用时4)
```

- 读法：`2 1 5 3 4` 表示"该工序有 2 台可选机器：机器 1 加工用时 5、机器 3 用时 4"。
- **同一道工序在不同机器上的加工时间不同**（5 vs 4）→ 机器相关的经典 FJSP，契合 §1 模型，无需退化处理。

Hurink et al. (1994)（从经典 JSP 基准改造：每个 base 实例 × 4 种柔性档）。56 个 base 实例（la01–la40, abz5–abz9, car1–car8, mt06/10/20 等，规模 6×6 → 30×10），同一 6×6 实例（即 mt06）的四档实测：

档位	目录	首行	每工序候选机器	柔性
S	`2a_Hurink_sdata`	`6 6 1`	恒为 1（`1 m t`）	无柔性，等价经典 JSP
E	`2b_Hurink_edata`	`6 6 1.15`	多为 1 台、个别 2 台	低柔性
R	`2c_Hurink_rdata`	`6 6 2`	2~4 台	中柔性
V	`2d_Hurink_vdata`	（更大）	最多	高柔性

- 候选机器数越多 → 调度自由度越高 → 同一实例可比较"柔性对算法的影响"；**推广到 Hurink 时按 s/e/r/v 四档分别报告。**

交货期：两套基准**均无交货期**（TWTE 需要 d_i）。按已定稿设计决策：**只取 Bagheri & Zandieh (2011) 法中的 SW 分支生成单一交货期 d_i，不用 EW**（原文 EW 是"交货期窗口"的上限，与本设计"单交货期 + 无权 E+T"无关）：

$d_i = SW_i \sim U\Big[\mu_i(1-\tfrac{R}{2}),\ \mu_i(1+\tfrac{R}{2})\Big],\qquad \mu_i=\Big(1+\tfrac{T\cdot I}{M}\Big)\sum_{j=1}^{J_i}\overline{PT}_{ij}$

- I = 作业数、M = 机器数（原文 Eq.27 明示）；$\overline{PT}_{ij}$ = 工件 i 第 j 道工序在**各候选机器上加工时间的均值**。
- **档位（已定稿）**：$T = 0.2$、$R=1.0$。理由：乘子 $(1+T\cdot I/M)$ 在 Brandimarte Mk01–Mk10 上 $I/M\in[0.67,4]$、乘子 $1+T\cdot I/M\in[1.13,1.80]$，贴近真实完工量级；$R=1.0$ 使 $d_i\in[0.5\mu_i,\ 1.5\mu_i]$（±50% 散布），保证同一实例中既有早到又有拖期、TWTE 两侧都激活，正是 §1.5"反向张力"所要求的观测窗口。
- （如后续需紧度敏感性，再扩展为 $T\in\{0.2,0.4,0.6\}\times R\in\{0.5,1.0\}$ 网格，本轮先固定单档。）

> **算例来源备注**（取代早前的"用户 xlsx"方案）：此前定的用户自有 xlsx（`D:\Study\研究生相关\研1\论文\FJSSP\Hybrid-Simheuristic-BiObj-Stoch-FJSSP\dataset\` 9 例）经核对是**机器无关加工时间**的退化 FJSP（加工时间 $pt[j][o]$ 不随机器改变、Machines Sequence 弃用），与 §1 经典 FJSP 及 §7.3 R1a"换慢机"的"换机改变 p"机制冲突。现改为 Brandimarte/Hurink 标准机器相关 FJSP；用户 xlsx 留作附录/降级备选，不再作为主算例。

---

13. 创新点清单与对现有工作的差异

#	创新点	定位	解决的问题 / 与现有差异
①	**交货期感知双向构造初始化**（双账本 GS-θ 路由 + 前向/反向构造 + 得分导向扰动 + 目标空间网格覆盖，见 §5）	阶段一	单方向预热导致的 TWTE 端塌陷；零迭代即覆盖两端+中间区；非凸前沿不塌陷
②	**双解码解耦评估**（左移算 makespan、右移空闲插入算 TWTE）	解码	一个解码拖累两目标；TWTE 端缺 idle 自由度
③	**交货期导向双向邻域**（左移 L1–L3 + 右移 R1 换慢机/顺序后移）	局部搜索	传统邻域只有左移、覆盖不到 TWTE 端
④	**成功率自适应算子选择 + 延迟触发**	局部搜索	固定概率算子不贴进化状态；前期浪费算力
⑤	**动态变异位数**（随 O_{total} 自适应，按迭代几何衰减 $\delta(gen)=\delta_{max}(\delta_{min}/\delta_{max})^{gen/gen_{max}}$）	变异	固定"变两位"的机械设定

> 说明：创新②是我把"同一染色体两套解码"显式提为一类机制（结论 C 的落实），也可视作创新③的姊妹设计。归一化统一为**逐代动态 min-max**（不含双归一化）。

---

14. 风险点与待确认问题（请重点审阅）

#	风险 / 待确认	说明与应对预案
R1	**逐代 min-max 早期范围窄**	早期 $f^{max}-f^{min}$ 小、归一化对极小扰动敏感。预案：阶段一先造两端（§5.3/§5.4）建立宽度；除零保护 ε 兜底；**存原始目标值、比较时现场归一化**（§4.1/§5.7），把比较严格限制在代内。**已确认接受。**
R2	**右移传播的正确性/实现复杂度**	右移空闲插入是 TWTE 正确建模的关键，但其"全局最优右移"实现有细节（逆时间序传播、不新增 tardiness）。设计里给了伪代码，落地时需小规模算例手工验证 TWTE 值。
R3	**makespan 与 TWTE 反向，R1 右移会不会反噬 makespan 端**	R1 只向右移、允许 f_1 略升，接受准则按"支配改进"；非支配排序会保留 makespan 端原解，理论上不塌陷前端；需消融实验验证。
R4	**交货期生成方式（已定稿）**	Brandimarte/Hurink 基准均无交货期，按已定稿设计决策 **只取 Bagheri & Zandieh (2011) 法 SW 分支生成单一 d_i、不用 EW**；紧度参数已定 $T = 0.2$、$R=1.0$（§12.1），避免对比不公平、消融不可比。

附：已定稿设计决策（历史记录，供追溯）

1. **预热求解器**：**已定稿为构造式初始化（§5，DABI）**，不使用任何迭代预热；§5.15 离散短程精修仅作可选增强。
2. **逐代 min-max 的早期不稳定**：**接受**，并采用"存原始目标值、比较时用当前代极值现场归一化"的口径（§4.1/§5.7），消除尺度漂移错配。
3. **随机加工时间**：已定稿为"阶段一固定、阶段二随机的 simheuristic 两阶段"，随机模型用**对数正态分布**（$UL\in\{0.1,0.25,0.5\}$，§9.1），迭代短仿真 $R=20$、最终长仿真 $R=10000$（§9.4），无需再搁置。

确认后即可进入编码实现。